

Digital Building Blocks

5

5.1 INTRODUCTION

Up to this point, we have examined the design of combinational and sequential circuits using Boolean equations, schematics, and HDLs. This chapter introduces more elaborate combinational and sequential building blocks used in digital systems. These blocks include arithmetic circuits, counters, shift registers, memory arrays, and logic arrays. These building blocks are not only useful in their own right but they also demonstrate the principles of hierarchy, modularity, and regularity. The building blocks are hierarchically assembled from simpler components, such as logic gates, multiplexers, and decoders. Each building block has a well-defined interface and can be treated as a black box when the underlying implementation is unimportant. The regular structure of each building block is easily extended to different sizes. In [Chapter 7](#), we use many of these building blocks to build a microprocessor.

5.2 ARITHMETIC CIRCUITS

Arithmetic circuits are the central building blocks of computers. Computers and digital logic perform many arithmetic functions: addition, subtraction, comparisons, shifts, multiplication, and division. This section describes hardware implementations for all of these operations.

5.2.1 Addition

Addition is one of the most common operations in digital systems. We first consider how to add two 1-bit binary numbers. We then extend to N -bit binary numbers. Adders also illustrate trade-offs between speed and complexity.

- 5.1 [Introduction](#)
- 5.2 [Arithmetic Circuits](#)
- 5.3 [Number Systems](#)
- 5.4 [Sequential Building Blocks](#)
- 5.5 [Memory Arrays](#)
- 5.6 [Logic Arrays](#)
- 5.7 [Summary](#)
- [Exercises](#)
- [Interview Questions](#)

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

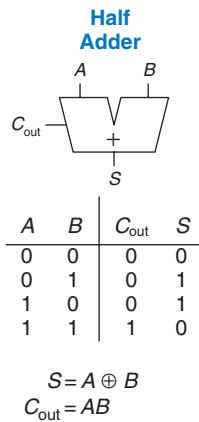


Figure 5.1 1-bit half adder

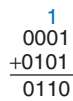


Figure 5.2 Carry bit

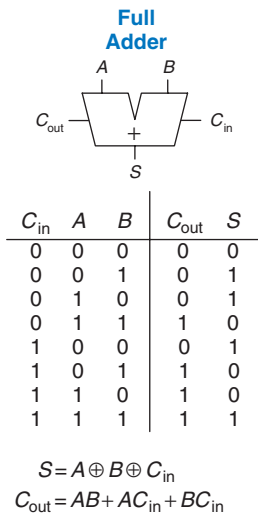


Figure 5.3 1-bit full adder

Half Adder

We begin by building a *half adder*. As shown in Figure 5.1, the half adder has two inputs, A and B, and two outputs, S and C_{out}. S is the sum of A and B. If A and B are both 1, S is 2, which cannot be represented with a single binary digit. Instead, it is indicated with a carry out C_{out} in the next column. The half adder can be built from an XOR gate and an AND gate.

In a multi-bit adder, C_{out} is added or *carried in* to the next most significant bit. For example, in Figure 5.2, the carry bit shown in blue is the output C_{out} of the first column of 1-bit addition and the input C_{in} to the second column of addition. However, the half adder lacks a C_{in} input to accept the C_{out} of the previous column. The *full adder*, described in the next section, solves this problem.

Full Adder

A *full adder*, introduced in Section 2.1, accepts the carry in C_{in} as shown in Figure 5.3. The figure also shows the output equations for S and C_{out}.

Carry Propagate Adder

An N-bit adder sums two N-bit inputs, A and B, and a carry in C_{in} to produce an N-bit result S and a carry out C_{out}. It is commonly called a *carry propagate adder* (CPA) because the carry out of one bit propagates into the next bit. The symbol for a CPA is shown in Figure 5.4; it is drawn just like a full adder except that A, B, and S are busses rather than single bits. Three common CPA implementations are called ripple-carry adders, carry-lookahead adders, and prefix adders.

Ripple-Carry Adder

The simplest way to build an N-bit carry propagate adder is to chain together N full adders. The C_{out} of one stage acts as the C_{in} of the next stage, as shown in Figure 5.5 for 32-bit addition. This is called a *ripple-carry adder*. It is a good application of modularity and regularity: the full adder module is reused many times to form a larger system. The ripple-carry adder has the disadvantage of being slow when N is large. S₃₁ depends on C₃₀, which depends on C₂₉, which depends on C₂₈, and so forth all the way back to C_{in}, as shown in blue in Figure 5.5. We say that the carry *ripples* through the carry chain. The delay of the adder,

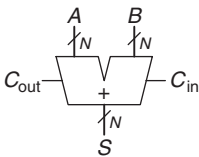


Figure 5.4 Carry propagate adder

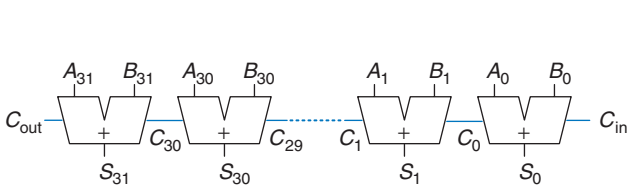


Figure 5.5 32-bit ripple-carry adder

t_{ripple} grows directly with the number of bits, as given in Equation. 5.1, where t_{FA} is the delay of a full adder.

$$t_{\text{ripple}} = Nt_{\text{FA}} \quad (5.1)$$

Carry-Lookahead Adder

The fundamental reason that large ripple-carry adders are slow is that the carry signals must propagate through every bit in the adder. A *carry-lookahead* adder (CLA) is another type of carry propagate adder that solves this problem by dividing the adder into *blocks* and providing circuitry to quickly determine the carry out of a block as soon as the carry in is known. Thus, it is said to *look ahead* across the blocks rather than waiting to ripple through all the full adders inside a block. For example, a 32-bit adder may be divided into eight 4-bit blocks.

CLAs use *generate* (G) and *propagate* (P) signals that describe how a column or block determines the carry out. The i th column of an adder is said to *generate* a carry if it produces a carry out independent of the carry in. The i th column of an adder is guaranteed to generate a carry C_i if A_i and B_i are both 1. Hence, G_i , the generate signal for column i , is calculated as $G_i = A_i \& B_i$. The column is said to *propagate* a carry if it produces a carry out whenever there is a carry in. The i th column will propagate a carry in, C_{i-1} , if either A_i or B_i is 1. Thus, $P_i = A_i \mid B_i$. Using these definitions, we can rewrite the carry logic for a particular column of the adder. The i th column of an adder will generate a carry out C_i if it either generates a carry, G_i , or propagates a carry in, P_iC_{i-1} . In equation form,

$$C_i = A_iB_i + (A_i + B_i)C_{i-1} = G_i + P_iC_{i-1} \quad (5.2)$$

The generate and propagate definitions extend to multiple-bit blocks. A block is said to generate a carry if it produces a carry out independent of the carry in to the block. The block is said to propagate a carry if it produces a carry out whenever there is a carry in to the block. We define 1-bit block generate and propagate signals, G_{ij} and P_{ij} , for blocks spanning columns i through j .

A block generates a carry if the most significant column generates a carry, or if the previous column generated a carry and the most significant column propagates it, and so forth. For example, the generate logic for a block spanning columns 3 through 0 is

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1G_0)) \quad (5.3)$$

A block propagates a carry if all the columns in the block propagate the carry. For example, the propagate logic for a block spanning columns 3 through 0 is

$$P_{3:0} = P_3P_2P_1P_0 \quad (5.4)$$

Schematics typically show signals flowing from left to right. Arithmetic circuits break this rule because the carries flow from right to left (from the least significant column to the most significant column). We keep the least significant column on the right and the most significant column on the left because that is how we're used to viewing and adding numbers.



Throughout the ages, people have used many devices to perform arithmetic. Toddlers count on their fingers (and some adults stealthily do too). The Chinese and Babylonians invented the abacus as early as 2400 BC. Slide rules, invented in 1630, were in use until the 1970's, when scientific hand calculators became prevalent. Computers and digital calculators are ubiquitous today. What will be next?

Using the block generate and propagate signals, we can quickly compute the carry out of the block, C_i , using the carry in to the block, C_{i-1} .

$$C_i = G_{i:j} + P_{i:j}C_{i-1} \quad (5.5)$$

Figure 5.6(a) shows a 32-bit carry-lookahead adder composed of eight 4-bit blocks. Each block contains a 4-bit ripple-carry adder and some lookahead logic to compute the carry out of the block given the carry in, as shown in Figure 5.6(b). The AND and OR gates needed to compute the column generate and propagate signals, G_i and P_i , using signals A_i and B_i are left out for brevity. Again, the carry-lookahead adder demonstrates modularity and regularity.

All of the CLA blocks compute the 1-bit column and block generate and propagate signals simultaneously. The critical path starts with computing G_0 and $G_{3:0}$ in the first CLA block. Then, C_{in} advances directly to C_{out} through the AND/OR gate in each block until the last. Specifically, after all of the column and block propagate and generate signals are calculated, C_{in} proceeds through the AND/OR gate to produce C_3 ; C_3 then proceeds through its block's AND/OR gate to produce C_7 ; C_7 proceeds through its block's AND/OR gate to produce C_{11} , and so on until

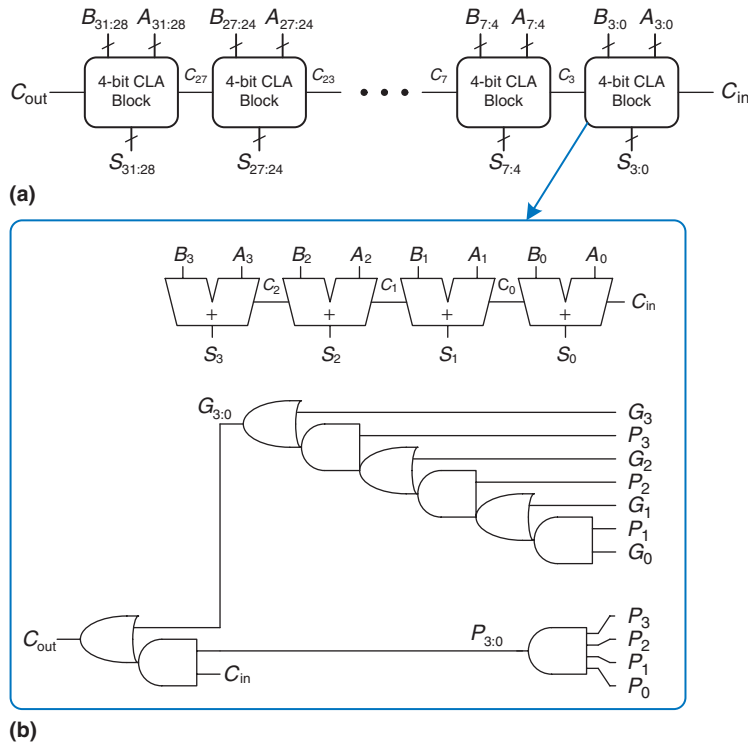


Figure 5.6 (a) 32-bit carry-lookahead adder (CLA), (b) 4-bit CLA block

C_{27} , the carry in to the last block. For a large adder, this is much faster than waiting for the carries to ripple through each consecutive bit of the adder. Finally, the critical path through the last block contains a short ripple-carry adder. Thus, an N -bit adder divided into k -bit blocks has a delay

$$t_{CLA} = t_{pg} + t_{pg_block} + \left(\frac{N}{k} - 1\right)t_{AND_OR} + kt_{FA} \quad (5.6)$$

where t_{pg} is the delay of the column propagate and generate gates (a single AND or OR gate) to generate P_i and G_i , t_{pg_block} is the delay to find the block propagate and generate signals P_{ij} and G_{ij} for a k -bit block, and t_{AND_OR} is the delay from C_{in} to C_{out} through the final AND/OR logic of the k -bit CLA block. For $N > 16$, the carry-lookahead adder is generally much faster than the ripple-carry adder. However, the adder delay still increases linearly with N .

Example 5.1 RIPPLE-CARRY ADDER AND CARRY-LOOKAHEAD ADDER DELAY

Compare the delays of a 32-bit ripple-carry adder and a 32-bit carry-lookahead adder with 4-bit blocks. Assume that each two-input gate delay is 100 ps and that a full adder delay is 300 ps.

Solution According to Equation 5.1, the propagation delay of the 32-bit ripple-carry adder is $32 \times 300 \text{ ps} = 9.6 \text{ ns}$.

The CLA has $t_{pg} = 100 \text{ ps}$, $t_{pg_block} = 6 \times 100 \text{ ps} = 600 \text{ ps}$, and $t_{AND_OR} = 2 \times 100 \text{ ps} = 200 \text{ ps}$. According to Equation 5.6, the propagation delay of the 32-bit carry-lookahead adder with 4-bit blocks is thus $100 \text{ ps} + 600 \text{ ps} + (32/4 - 1) \times 200 \text{ ps} + (4 \times 300 \text{ ps}) = 3.3 \text{ ns}$, almost three times faster than the ripple-carry adder.

Prefix Adder*

Prefix adders extend the generate and propagate logic of the carry-lookahead adder to perform addition even faster. They first compute G and P for pairs of columns, then for blocks of 4, then for blocks of 8, then 16, and so forth until the generate signal for every column is known. The sums are computed from these generate signals.

In other words, the strategy of a prefix adder is to compute the carry in C_{i-1} for each column i as quickly as possible, then to compute the sum, using

$$S_i = (A_i \oplus B_i) \oplus C_{i-1} \quad (5.7)$$

Define column $i = -1$ to hold C_{in} , so $G_{-1} = C_{in}$ and $P_{-1} = 0$. Then, $C_{i-1} = G_{i-1;-1}$ because there will be a carry out of column $i - 1$ if the block spanning columns $i - 1$ through -1 generates a carry. The generated carry

Early computers used ripple-carry adders because components were expensive and ripple-carry adders used the least hardware. Virtually all modern PCs use prefix adders on critical paths because transistors are now cheap and speed is of great importance.

is either generated in column $i - 1$ or generated in a previous column and propagated. Thus, we rewrite Equation 5.7 as

$$S_i = (A_i \oplus B_i) \oplus G_{i-1:-1} \quad (5.8)$$

Hence, the main challenge is to rapidly compute all the block generate signals $G_{-1:-1}$, $G_{0:-1}$, $G_{1:-1}$, $G_{2:-1}$, ..., $G_{N-2:-1}$. These signals, along with $P_{-1:-1}$, $P_{0:-1}$, $P_{1:-1}$, $P_{2:-1}$, ..., $P_{N-2:-1}$, are called *prefixes*.

Figure 5.7 shows an $N = 16$ -bit prefix adder. The adder begins with a *precomputation* to form P_i and G_i for each column from A_i and B_i using AND and OR gates. It then uses $\log_2 N = 4$ levels of black cells to form the prefixes of $G_{i:j}$ and $P_{i:j}$. A black cell takes inputs from the upper part of a block spanning bits $i:k$ and from the lower part spanning bits $k - 1:j$. It combines these parts to form generate and propagate signals for the entire block spanning bits $i:j$ using the equations

$$G_{i:j} = G_{i:k} + P_{i:k}G_{k-1:j} \quad (5.9)$$

$$P_{i:j} = P_{i:k} P_{k-1:j} \quad (5.10)$$

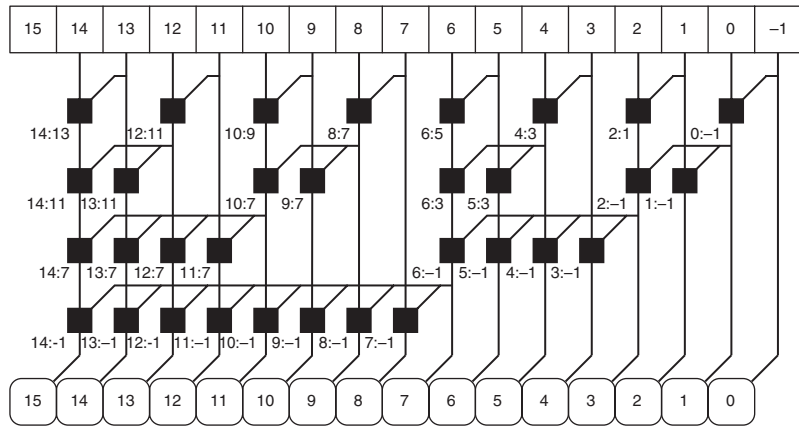
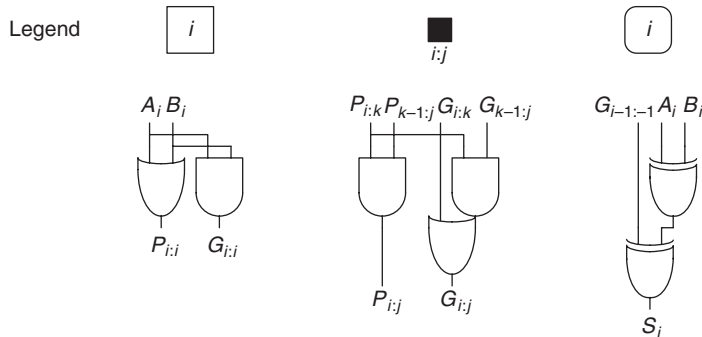


Figure 5.7 16-bit prefix adder



In other words, a block spanning bits $i:j$ will generate a carry if the upper part ($i:k$) generates a carry or if the upper part propagates a carry that is generated in the lower part ($k-1:j$). The block will propagate a carry if both the upper and lower parts propagate the carry. Finally, the prefix adder computes the sums using Equation 5.8.

In summary, the prefix adder achieves a delay that grows logarithmically rather than linearly with the number of columns in the adder. This speedup is significant, especially for adders with 32 or more bits, but it comes at the expense of more hardware than a simple carry-lookahead adder. The network of black cells is called a *prefix tree*.

The general principle of using prefix trees to perform computations in time that grows logarithmically with the number of inputs is a powerful technique. With some cleverness, it can be applied to many other types of circuits (see, for example, Exercise 5.7).

The critical path for an N -bit prefix adder involves the precomputation of P_i and G_i followed by $\log_2 N$ stages of black prefix cells to obtain all of the prefixes. $G_{i-1:-1}$ then proceeds through the final XOR gate at the bottom to compute S_i . Mathematically, the delay of an N -bit prefix adder is

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR} \quad (5.11)$$

where t_{pg_prefix} is the delay of a black prefix cell.

Notice that the delay of the final (sum) box in the prefix adder is one XOR delay (not two) because the computation of $A_i \oplus B_i$ already occurred earlier because A_i and B_i were available.

Example 5.2 PREFIX ADDER DELAY

Compute the delay of a 32-bit prefix adder. Assume that each two-input gate delay is 100 ps.

Solution The propagation delay of each black prefix cell t_{pg_prefix} is 200 ps (i.e., two gate delays). Thus, using Equation 5.11, the propagation delay of the 32-bit prefix adder is $100 \text{ ps} + \log_2(32) \times 200 \text{ ps} + 100 \text{ ps} = 1.2 \text{ ns}$, which is about three times faster than the carry-lookahead adder and eight times faster than the ripple-carry adder from Example 5.1. In practice, the benefits are not quite this great, but prefix adders are still substantially faster than the alternatives.

Putting It All Together

This section introduced the half adder, full adder, and three types of carry propagate adders: ripple-carry, carry-lookahead, and prefix adders. Faster adders require more hardware and therefore are more expensive and power-hungry. These trade-offs must be considered when choosing an appropriate adder for a design.

Hardware description languages provide the $+$ operation to specify a CPA. Modern synthesis tools select among many possible implementations,

HDL Example 5.1 CARRY PROPAGATE ADDER (CPA)

SystemVerilog

```
module adder #(parameter N = 8)
  (input  logic [N-1:0] a, b,
   input  logic      cin,
   output logic [N-1:0] s,
   output logic      cout);

  assign {cout, s} = a + b + cin;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity adder is
  generic(N: integer := 8);
  port(a, b: in  STD_LOGIC_VECTOR(N-1 downto 0);
       cin: in  STD_LOGIC;
       s:   out STD_LOGIC_VECTOR(N-1 downto 0);
       cout: out STD_LOGIC);
end;

architecture synth of adder is
  signal result: STD_LOGIC_VECTOR(N downto 0);
begin
  result <= ("0" & a) + ("0" & b) + cin;
  s      <= result(N-1 downto 0);
  cout   <= result(N);
end;
```

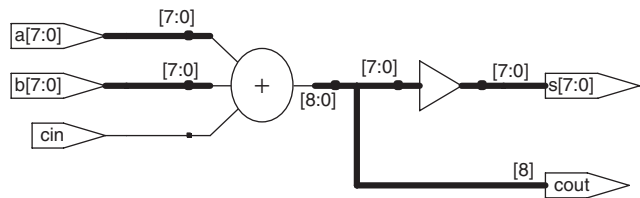


Figure 5.8 Synthesized adder

choosing the cheapest (smallest) design that meets the speed requirements. This greatly simplifies the designer’s job. HDL Example 5.1 describes a CPA with carries in and out, and Figure 5.8 shows the resulting hardware.

5.2.2 Subtraction

Recall from Section 1.4.6 that adders can add positive and negative numbers using two’s complement number representation. Subtraction is almost as easy: flip (i.e., reverse) the sign of the second number, then add. Flipping the sign of a two’s complement number is done by inverting the bits and adding 1.

To compute $Y = A - B$, first create the two’s complement of B : invert the bits of B to obtain $\bar{B}$ and add 1 to get $-B = \bar{B} + 1$. Add this quantity to A to get $Y = A + \bar{B} + 1 = A - B$. This sum can be performed with a single CPA by adding $A + \bar{B}$ with $C_{in} = 1$. Figure 5.9 shows the symbol for a subtractor and the underlying hardware for performing $Y = A - B$. HDL Example 5.2 describes a subtractor, and Figure 5.10 shows the resulting hardware.

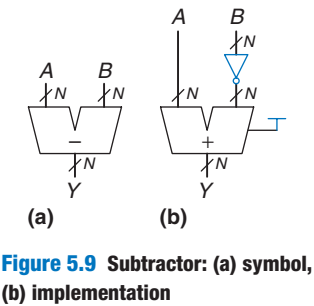


Figure 5.9 Subtractor: (a) symbol, (b) implementation

HDL Example 5.2 SUBTRACTOR

SystemVerilog

```
module subtractor #(parameter N = 8)
    (input  logic [N-1:0] a, b,
     output logic [N-1:0] y);

    assign y = a - b;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity subtractor is
    generic(N: integer := 8);
    port(a, b: in  STD_LOGIC_VECTOR(N-1 downto 0);
         y:      out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture synth of subtractor is
begin
    y <= a - b;
end;
```

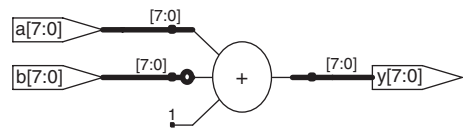


Figure 5.10 Synthesized subtractor

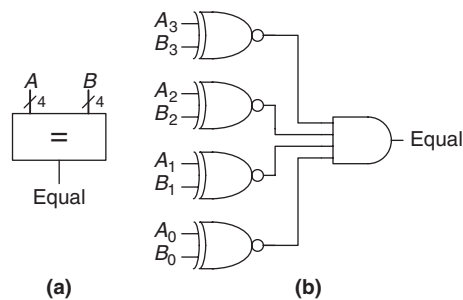


Figure 5.11 4-bit equality comparator: (a) symbol, (b) implementation

5.2.3 Comparators

A *comparator* determines whether two binary numbers are equal or if one is greater or less than the other. A comparator receives two N -bit binary numbers A and B and outputs a 1-bit comparison result.

An *equality comparator* produces a single output, indicating whether A is equal to B ($A == B$). A *magnitude comparator* produces one or more outputs, indicating the relative values of A and B .

The equality comparator is the simpler piece of hardware. Figure 5.11 shows the symbol and implementation of a 4-bit equality comparator. It first checks to determine whether the corresponding bits in each column of A and B are equal, using XNOR gates. The numbers are equal if all of the columns are equal.

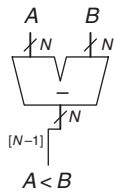


Figure 5.12 *N*-bit signed comparator

Magnitude comparison of signed numbers is usually done by computing $A - B$ and looking at the sign (most significant bit) of the result as shown in [Figure 5.12](#). If the result is negative (i.e., the sign bit is 1), then A is less than B . Otherwise, A is greater than or equal to B . This comparator, however, functions incorrectly upon overflow. Exercises 5.9 and 5.10 explore this limitation and how to fix it. [HDL Example 5.3](#) shows how to use various comparison operations for unsigned numbers, and [Figure 5.13](#) shows the resulting hardware.

HDL Example 5.3 COMPARATORS

SystemVerilog

```
module comparators #(parameter N = 8)
    (input  logic [N-1:0] a, b,
     output logic eq, neq, lt, lte, gt, gte);

    assign eq  = (a == b);
    assign neq = (a != b);
    assign lt  = (a < b);
    assign lte = (a <= b);
    assign gt  = (a > b);
    assign gte = (a >= b);
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity comparators is
    generic(N: integer := 8);
    port(a, b: in STD_LOGIC_VECTOR(N-1 downto 0);
         eq, neq, lt, lte, gt, gte: out STD_LOGIC);
end;

architecture synth of comparators is
begin
    eq <= '1' when (a = b) else '0';
    neq <= '1' when (a /= b) else '0';
    lt <= '1' when (a < b) else '0';
    lte <= '1' when (a <= b) else '0';
    gt <= '1' when (a > b) else '0';
    gte <= '1' when (a >= b) else '0';
end;
```

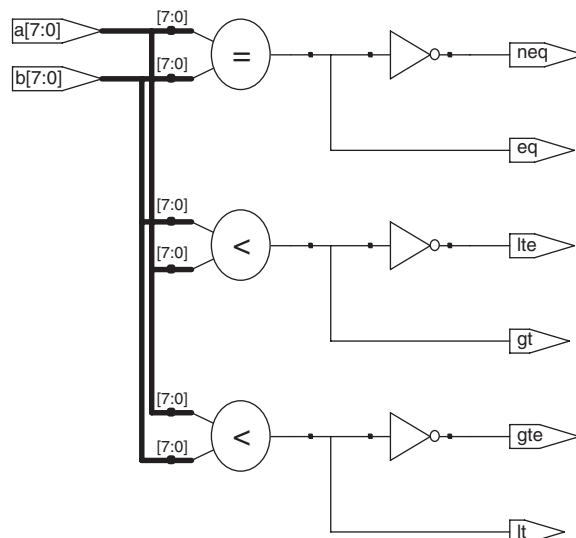


Figure 5.13 Synthesized comparators

Table 5.1 ALU operations

$ALUControl_{1:0}$	Function
00	Add
01	Subtract
10	AND
11	OR

5.2.4 ALU

An *Arithmetic/Logical Unit* (ALU) combines a variety of mathematical and logical operations into a single unit. For example, a typical ALU might perform addition, subtraction, AND, and OR operations. The ALU forms the heart of most computer systems.

Figure 5.14 shows the symbol for an N -bit ALU with N -bit inputs and outputs. The ALU receives a 2-bit control signal $ALUControl$ that specifies which function to perform. Control signals will generally be shown in blue to distinguish them from the data. Table 5.1 lists typical functions that the ALU can perform.

Figure 5.15 shows an implementation of the ALU. The ALU contains an N -bit adder and N 2-input AND and OR gates. It also contains inverters

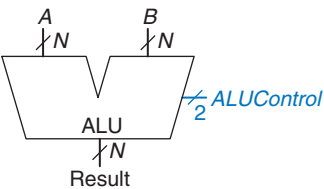


Figure 5.14 ALU symbol

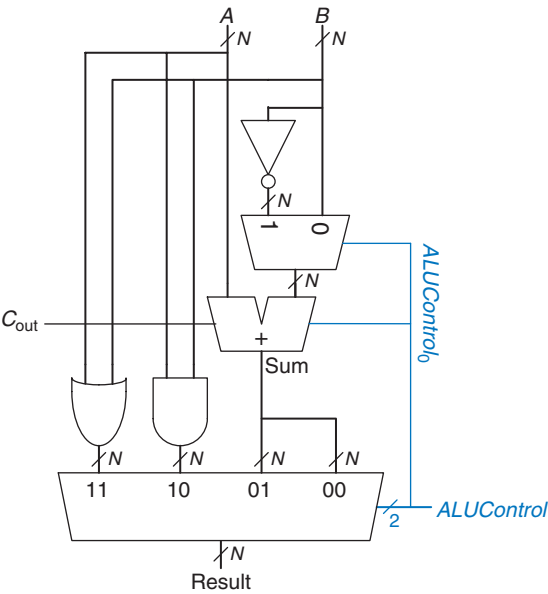


Figure 5.15 N -bit ALU

Table 5.2 Signed and unsigned comparisons

Comparison	Signed	Unsigned
$=$	Z	Z
$\neq$	$\bar{Z}$	$\bar{Z}$
$<$	$N \oplus V$	$\bar{C}$
$\leq$	$Z + (N \oplus V)$	$Z + \bar{C}$
$>$	$\bar{Z} \bullet (\bar{N} \oplus \bar{V})$	$\bar{Z} \bullet C$
$\geq$	$(\bar{N} \oplus \bar{V})$	C

two same signed numbers produces a result with the opposite sign. So, V (oVerflow) is asserted when all three of the following conditions are true: (1) the ALU is performing addition or subtraction ($ALUControl_1 = 0$), (2) A and Sum have opposite signs, as detected by the XOR gate, and (3) overflow is possible. That is, as detected by the XNOR gate, either A and B have the same sign and the adder is performing addition ($ALUControl_0 = 0$) or A and B have opposite signs and the adder is performing subtraction ($ALUControl_0 = 1$). The 3-input AND gate detects when all three conditions are true and asserts V .

The ALU flags can also be used for comparisons, as shown in Table 5.2. To compare inputs A and B , the ALU computes $A - B$ and looks at the flags. If Z is asserted, the result is 0, so $A = B$. Otherwise, A is not equal to B .

Magnitude comparison is messier and depends on whether the numbers are signed or unsigned. For example, to determine $A < B$, we compute $A - B$ and check whether the result is negative. If the numbers are unsigned, the result is negative if there is no carry out.¹ If the numbers are signed, we can't rely on the carry because small negative numbers are represented in the same way as large positive unsigned numbers. Instead, we simply compute $A - B$ and see whether the answer is negative, indicated by the N flag. However, if overflow occurs, the N flag will be incorrect. Hence, A is less than B if $N \oplus V$ (in other words, if the answer is negative and there is no overflow or if the answer is positive but overflow occurred). In summary, let us define L (the *Less than signal*) to be true if $A < B$. For unsigned numbers, $L = \bar{C}$. For signed

¹ You can check this by trying some numbers. Alternatively, note that reversing the sign (i.e., taking the two's complement) of N -bit numbers for subtraction produces $-B = \bar{B} + 1 = 2^N - B$. Now, $A + (-B) = 2^N + A - B$. This will have a carry out (a 1 in column N) if $A \geq B$ and, hence, no carry out if $A < B$.

numbers, $L = N \oplus V$. The remainder of the checks are easier. Less than or equal to ($\leq$) is L OR Z because L indicates less than and Z indicates equal to. Greater than or equal to ($\geq$) is the opposite of less than: $\bar{L}$. Greater than ($>$) is indicated by greater than or equal to, but not equal: $\bar{L}$ AND $\bar{Z}$.

Example 5.3 COMPARISONS

Consider 4-bit numbers $A = 1111$ and $B = 0010$. Determine whether $A < B$, first interpreting the numbers as unsigned (15 and 2) and then as signed (−1 and 2).

Solution Compute $A - B = A + \bar{B} + 1 = 1111 + 1101 + 1 = 11101$. The carry out C is 1, as shown in blue. The N flag is 1, as shown in italics. The V flag is 0 because the result has the same sign bit as A . The Z flag is 0 because the answer is not 0000.

For unsigned comparison, $L = \sim C = 0$ because 15 is not less than 2. For signed comparison, $L = N \oplus V = 1$ because −1 is less than 2.

Certain ALUs also implement an instruction called *set if less than* (SLT). When $A < B$, *Result* = 1. Otherwise, *Result* = 0. This is convenient for computers that do not have access to ALU flags because it essentially stores flag information in the result. SLT typically treats inputs as signed. Another flavor (SLTU) treats inputs as unsigned. Many variations on this basic ALU exist that support other functions, such as NOT, XOR, or XNOR. The HDL for an N -bit ALU, including versions that support SLT and output flags, is left to Exercises 5.11 to 5.14.

Example 5.4 EXPANDING THE ALU TO HANDLE SLT

Expand the ALU to handle the set if less than (SLT) operation.

Solution To add another function to the ALU, we must expand the multiplexer to have five inputs. We determine if A is less than B by performing $A - B$; if the result is negative, A is less than B . Table 5.3 shows the updated *ALUControl* signal for handling SLT, and Figure 5.18(a) shows the expanded circuit, with changes highlighted in blue (for control signals) and black. We use $ALUControl = 101$ for the SLT operation and take advantage of the fact that making $ALUControl_0 = 1$ causes the adder to perform $A - B$. When $Sum_{N-1} = 1$, the result of $A - B$ is negative, and A is less than B . So, we zero-extend Sum_{N-1} and feed it into the 101 multiplexer input to complete the SLT operation. Note, however, that this implementation does not account for overflow. When overflow occurs, *Sum* will have the incorrect sign. So, we XOR the sign bit of *Sum* with V , the overflow signal, to correctly indicate a negative *Sum*, as shown in Figure 5.18(b).

Table 5.3 ALU expanded operation for SLT

<i>ALUControl</i> _{2:0}	Function
000	Add
001	Subtract
010	AND
011	OR
101	SLT

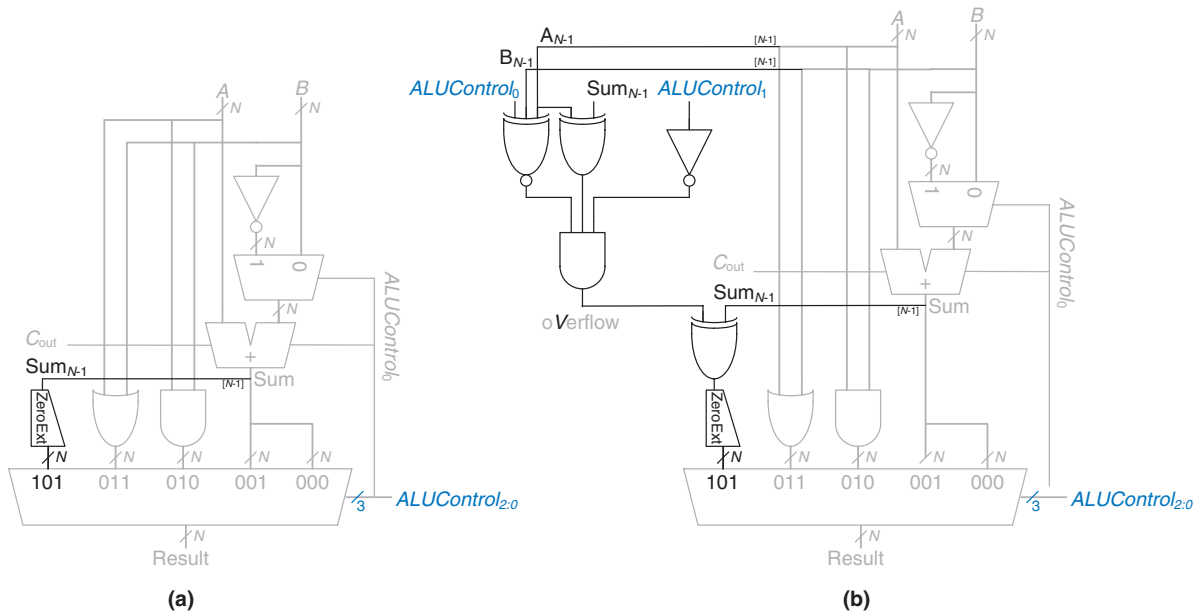


Figure 5.18 ALU expanded to support SLT (a) not accounting for overflow, (b) accounting for overflow

5.2.5 Shifters and Rotators

Shifters and *rotators* move bits and multiply or divide by powers of 2. As the name implies, a shifter shifts a binary number left or right by a specified number of positions. Several kinds of commonly used shifters exist:

- **Logical shifter**—shifts the number to the left or right and fills empty spots with 0's.

Example: 11001 >> 2 = 00110; 11001 << 2 = 00100

- **Arithmetic shifter**—is the same as a logical shifter, but on right shifts fills the most significant bits with a copy of the old most significant bit (msb). This is useful for multiplying and dividing signed numbers (see Sections 5.2.6 and 5.2.7). Arithmetic shift left is the same as logical shift left.

Example: 11001 >>> 2 = 11110; 11001 << 2 = 00100. The operators <<, >>, and >>> typically indicate shift left, logical shift right, and arithmetic shift right, respectively.

- **Rotator**—rotates a number in a circle such that empty spots are filled with bits shifted off the other end.

Example: 11001 ROR 2 = 01110; 11001 ROL 2 = 00111. ROR = rotate right; ROL = rotate left.

An N -bit shifter can be built from N N :1 multiplexers. The input is shifted by 0 to $N-1$ bits, depending on the value of the $\log_2 N$ -bit select lines. Figure 5.19 shows the symbol and hardware of 4-bit shifters. Depending on the value of the 2-bit shift amount $shamt_{1:0}$, the output Y receives the input A shifted by 0 to 3 bits. For all shifters, when $shamt_{1:0} = 00$, $Y = A$. Exercise 5.22 covers rotator designs.

A left shift is a special case of multiplication. A left shift by N bits multiplies the number by 2^N . For example, $000011_2 \ll 4 = 110000_2$ is equivalent to $3_{10} \times 2^4 = 48_{10}$.

An arithmetic right shift is a special case of division. An arithmetic right shift by N bits divides the number by 2^N . For example, $11100_2 \ggg 2 = 11111_2$ is equivalent to $-4_{10}/2^2 = -1_{10}$.

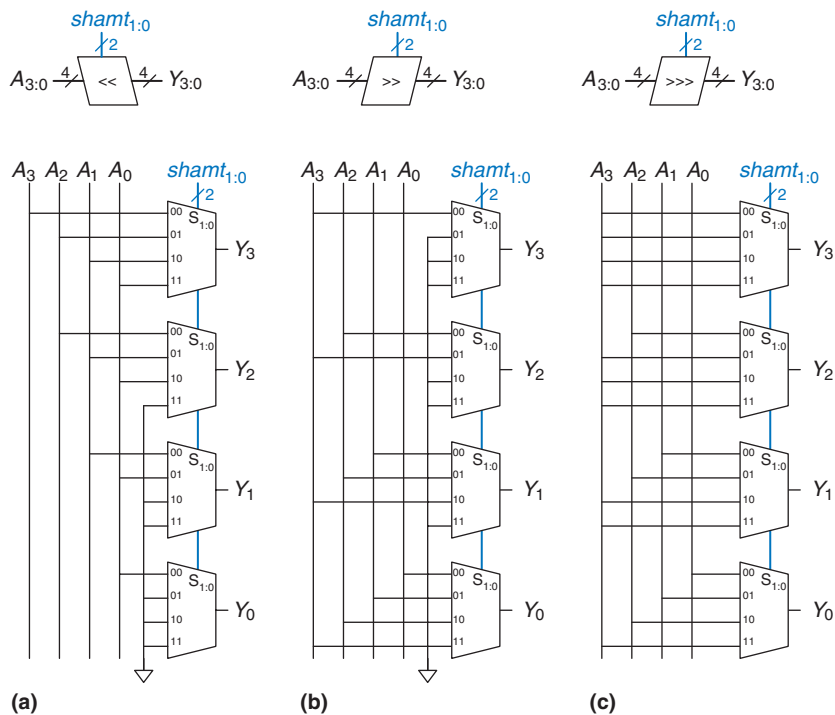


Figure 5.19 4-bit shifters: (a) shift left, (b) logical shift right, (c) arithmetic shift right

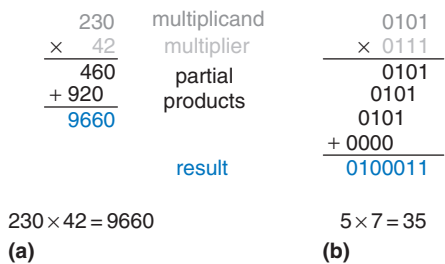


Figure 5.20 Multiplication:
(a) decimal, (b) binary

5.2.6 Multiplication*

Multiplication of unsigned binary numbers is similar to decimal multiplication but involves only 1’s and 0’s. Figure 5.20 compares multiplication in decimal and binary. In both cases, *partial products* are formed by multiplying a single digit of the multiplier with the entire multiplicand. The shifted partial products are summed to form the result.

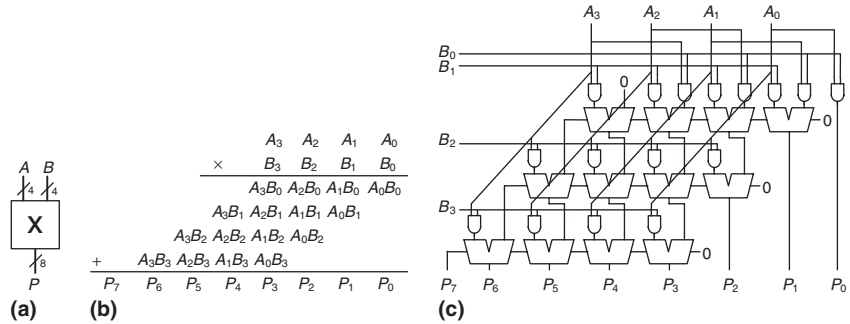
In general, an $N \times N$ multiplier multiplies two N -bit numbers and produces a $2N$ -bit result. The partial products in binary multiplication are either the multiplicand or all 0’s. Multiplication of 1-bit binary numbers is equivalent to the AND operation, so AND gates are used to form the partial products.

Signed and unsigned multiplication differ. For example, consider $0xFE \times 0xFD$. If these 8-bit numbers are interpreted as signed integers, they represent -2 and -3 , so the 16-bit product is $0x0006$. If these numbers are interpreted as unsigned integers, the 16-bit product is $0xFB06$. Notice that, in either case, the least significant byte is $0x06$.

Figure 5.21 shows the symbol, function, and implementation of an unsigned 4×4 multiplier. The unsigned multiplier receives the multiplicand and multiplier, A and B , and produces the product P . Figure 5.21(b) shows how partial products are formed. Each partial product is a single multiplier bit (B_3, B_2, B_1 , or B_0) AND the multiplicand bits (A_3, A_2, A_1, A_0). With N -bit operands, there are N partial products and $N - 1$ stages of 1-bit adders. For example, for a 4×4 multiplier, the partial product of the first row is B_0 AND (A_3, A_2, A_1, A_0). This partial product is added to the shifted second partial product, B_1 AND (A_3, A_2, A_1, A_0). Subsequent rows of AND gates and adders form and add the remaining partial products.

The HDL for signed and unsigned multipliers is in HDL Example 4.33. As with adders, many different multiplier designs with different speed/cost trade-offs exist. Synthesis tools may pick the most appropriate design given the timing constraints.

Figure 5.21 4×4 multiplier:
(a) symbol, (b) function, (c)
implementation



A *multiply accumulate* operation multiplies two numbers and adds them to a third number—typically, the accumulated value. These operations, also called MACs, are often used in *digital signal processing* (DSP) algorithms such as the Fourier transform, which requires a summation of products.

5.2.7 Division*

Binary division can be performed using the following algorithm for N -bit unsigned numbers in the range $[0, 2^N - 1]$:

```

R' = 0
for i = N-1 to 0
    R = {R' << 1, Ai}
    D = R - B
    if D < 0 then Qi = 0, R' = R // R < B
    else      Qi = 1, R' = D // R ≥ B
R = R'

```

The *partial remainder* R is initialized to 0 ($R' = 0$), and the most significant bit of the dividend A becomes the least significant bit of R ($R = \{R' \ll 1, A_i\}$). The divisor B is subtracted from this partial remainder to determine whether it fits ($D = R - B$). If the difference D is negative (i.e., the sign bit of D is 1), then the quotient bit Q_i is 0 and the difference is discarded. Otherwise, Q_i is 1, and the partial remainder is updated to be the difference. In any event, the partial remainder is then doubled (left-shifted by one column), the next most significant bit of A becomes the least significant bit of R , and the process repeats. The result satisfies $\frac{A}{B} = Q + \frac{R}{B}$.

Figure 5.22 shows a schematic of a 4-bit array divider. The divider computes A/B and produces a quotient Q and a remainder R . The legend shows the symbol and schematic for each block in the array

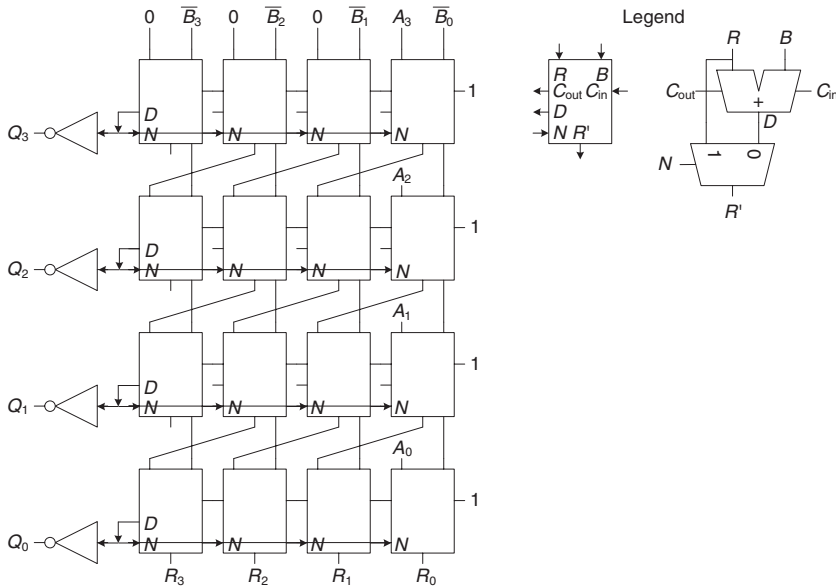


Figure 5.22 Array divider

divider. Each row performs one iteration of the division algorithm. Specifically, each row calculates the difference $D = R - B$. (Recall that $R + \bar{B} + 1 = R - B$). The multiplexer select signal, N (for Negative), receives 1 when a row's difference D is negative. So N is driven by the most significant bit of D , which is 1 when the difference is negative. Each quotient bit (Q_i) is 0 when D is negative and 1 otherwise. The multiplexer passes R to the next row if the difference is negative and D otherwise. The following row shifts the new partial remainder left by one bit, appends the next most significant bit of A , and then repeats the process.

The delay of an N -bit array divider increases proportionally to N^2 because the carry must ripple through all N stages in a row before the sign is determined and the multiplexer selects R or D . This repeats for all N rows. Division is a slow and expensive operation in hardware; therefore, it should be used as infrequently as possible.

5.2.8 Further Reading

Computer arithmetic could be the subject of an entire text. *Digital Arithmetic*, by Ercegovac and Lang, is an excellent overview of the entire field. *CMOS VLSI Design*, by Weste and Harris, covers high-performance circuit designs for arithmetic operations.

5.3 NUMBER SYSTEMS

Computers operate on both integers and fractions. So far, we have only considered representing signed or unsigned integers, as introduced in [Section 1.4](#). This section introduces fixed- and floating-point number systems that can represent rational numbers. Fixed-point numbers are analogous to decimals; some of the bits represent the integer part, and the rest represent the fraction. Floating-point numbers are analogous to scientific notation, with a mantissa and an exponent.

5.3.1 Fixed-Point Number Systems

Fixed-point notation has an implied *binary point* between the integer and fraction bits, analogous to the decimal point between the integer and fraction digits of an ordinary decimal number. For example, [Figure 5.23\(a\)](#) shows a fixed-point number with four integer bits and four fraction bits. [Figure 5.23\(b\)](#) shows the implied binary point in blue, and [Figure 5.23\(c\)](#) shows the equivalent decimal value.

Signed fixed-point numbers can use either two's complement or sign/magnitude notation. [Figure 5.24](#) shows the fixed-point representation of -2.375 using both notations with four integer and four fraction bits. The implicit binary point is shown in blue for clarity. In sign/magnitude form, the most significant bit is used to indicate the sign. The two's complement representation is formed by inverting the bits of the absolute value and adding a 1 to the least significant (rightmost) bit. In this case, the least significant bit position is in the 2^{-4} column.

Like all binary number representations, fixed-point numbers are just a collection of bits. There is no way of knowing the existence of the binary point except through agreement of those people interpreting the number.

In general, we use $Ua.b$ to denote an unsigned fixed-point number with a integer bits and b fraction bits. $Qa.b$ denotes a signed (two's complement) fixed point number with a integer bits (including the sign bit) and b fractional bits.

- (a) 01101100
 (b) 0110.1100
 (c) $2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$

Figure 5.23 Fixed-point notation of 6.75 with four integer bits and four fraction bits

- (a) 0010.0110
 (b) 1010.0110
 (c) 1101.1010

Figure 5.24 Fixed-point representation of -2.375 : (a) absolute value, (b) sign and magnitude, (c) two's complement

Example 5.5 ARITHMETIC WITH FIXED-POINT NUMBERS

Compute $0.75 + (-0.625)$ using Q4.4 fixed-point numbers.

Solution First, convert 0.625, the magnitude of the second number, to fixed-point binary notation. $0.625 \geq 2^{-1}$, so there is a 1 in the 2^{-1} column, leaving $0.625 - 0.5 = 0.125$. Because $0.125 < 2^{-2}$, there is a 0 in the 2^{-2} column. Because

0000.1010

1111.0101

+

1

1111.0110

Binary Magnitude

One's Complement

Add 1

Two's Complement

0000.1100

+

1111.0110

10000.0010

0.75

+

(-0.625)

0.125

(a)

(b)

Figure 5.25 Fixed-point two's complement conversion

Figure 5.26 Addition: (a) binary fixed-point, (b) decimal equivalent

$0.125 \geq 2^{-3}$, there is a 1 in the 2^{-3} column, leaving $0.125 - 0.125 = 0$. Thus, there must be a 0 in the 2^{-4} column. Putting this all together, $0.625_{10} = 0000.1010_2$.

Use two's complement representation for signed numbers so that addition works correctly. Figure 5.25 shows the conversion of -0.625 to fixed-point two's complement notation.

Figure 5.26 shows the fixed-point binary addition and the decimal equivalent for comparison. Note that the leading 1 in the binary fixed-point addition of Figure 5.26(a) is discarded from the 8-bit result.

5.3.2 Floating-Point Number Systems*

Floating-point numbers are analogous to scientific notation. They circumvent the limitation of having a constant number of integer and fraction bits, allowing the representation of very large and very small numbers. Like scientific notation, floating-point numbers have a *sign*, *mantissa* (M), *base* (B), and *exponent* (E), as shown in Figure 5.27. For example, the number 4.1×10^3 is the decimal scientific notation for 4100. It has a mantissa of 4.1, a base of 10, and an exponent of 3. The decimal point *floats* to the position right after the most significant digit. Floating-point numbers are base 2 with a binary mantissa. 32 bits are used to represent 1 sign bit, 8 exponent bits, and 23 mantissa bits.

Fixed-point number systems are commonly used in digital signal processing (DSP), graphics, and machine learning applications because the computations are faster and consume less power than they would in floating-point systems. Q1.15 (also known as Q15) is the most common format, storing signed numbers in the range $(-1, 1)$ with 15 bits of precision. Q1.31 (also called just Q31) is sometimes used for higher-precision intermediate results, such as in a Fast Fourier Transform. U8.8 is sometimes used for sensor readings sampled by analog/digital converters (ADCs). Note that all of these formats pack into 16- or 32-bit words for efficient storage in computer memories, which are typically a power of 2 in width.

Example 5.6 32-BIT FLOATING-POINT NUMBERS

Show the floating-point representation of the decimal number 228.

Solution First, convert the decimal number into binary: $228_{10} = 11100100_2 = 1.11001_2 \times 2^7$. Figure 5.28 shows the 32-bit encoding, which will be modified later for efficiency. The sign bit is positive (0), the 8 exponent bits give the value 7, and the remaining 23 bits are the mantissa.

$\pm M \times B^E$

Figure 5.27 Floating-point numbers

Figure 5.28 32-bit floating-point version 1

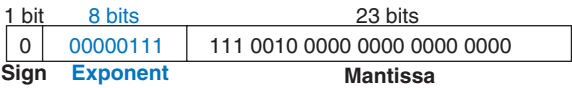
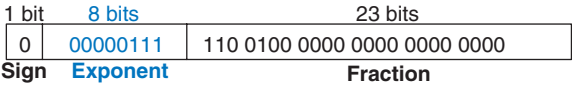


Figure 5.29 32-bit floating-point version 2



As may be apparent, there are many reasonable ways to represent floating-point numbers. For many years, computer manufacturers used incompatible floating-point formats. Results from one computer could not directly be interpreted by another computer.

The Institute of Electrical and Electronics Engineers (IEEE) solved this problem by creating the *IEEE 754 Floating-Point Standard* in 1985, that defines floating-point numbers. This floating-point format is now almost universally used and is the one discussed in this section.

In binary floating-point, the first bit of the mantissa (to the left of the binary point) is always 1 and therefore need not be stored. It is called the *implicit leading one*. Figure 5.29 shows the modified floating-point representation of $228_{10} = 11100100_2 \times 2^0 = 1.11001_2 \times 2^7$. The implicit leading one is not included in the 23-bit mantissa for efficiency. Only the fraction bits are stored. This frees up an extra bit for useful data.

We make one final modification to the exponent field. The exponent needs to represent both positive and negative exponents. To do so, floating-point uses a *biased* exponent, which is the original exponent plus a constant bias. 32-bit floating-point uses a bias of 127. For example, for the exponent 7, the biased exponent is $7 + 127 = 134 = 10000110_2$. For the exponent -4 , the biased exponent is: $-4 + 127 = 123 = 01111011_2$. Figure 5.30 shows $1.11001_2 \times 2^7$ represented in floating-point notation with an implicit leading one and a biased exponent of 134 ($7 + 127$). This notation conforms to the IEEE 754 floating-point standard.

Special Cases: 0, ±∞, and NaN

The IEEE floating-point standard has special cases to represent numbers such as zero, infinity, and illegal results. For example, representing the number zero is problematic in floating-point notation because of the implicit leading one. Special codes with exponents of all 0's or all 1's are reserved for these special cases. Table 5.4 shows the floating-point representations of 0, ±∞, and NaN. As with sign/magnitude numbers, floating-point has both positive and negative 0. NaN is used for numbers that don't exist, such as $\sqrt{-1}$ or $\log_2(-5)$.

Single-, Double-, and Quad-Precision Formats

So far, we have examined 32-bit floating-point numbers. This format is also called *single-precision*, *single*, or *float*. The IEEE 754 standard also

Figure 5.30 IEEE 754 floating-point notation

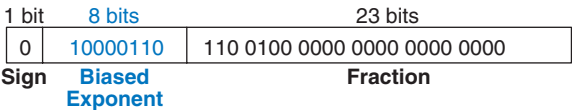


Table 5.4 IEEE 754 floating-point notations for 0, $\pm\infty$, and NaN

Number	Sign	Exponent	Fraction
0	X	00000000	000000000000000000000000
∞	0	11111111	000000000000000000000000
$-\infty$	1	11111111	000000000000000000000000
NaN	X	11111111	Non-zero

Table 5.5 Floating-point formats

Format	Total Bits	Sign Bits	Exponent Bits	Fraction Bits	Bias
Single	32	1	8	23	127
Double	64	1	11	52	1023
Quad	128	1	15	112	16363

defines 64-bit *double-precision* numbers (also called *doubles*) and 128-bit *quadruple-precision* numbers (also called *quads*) that provide greater precision and greater range. Table 5.5 shows the number of bits used for the fields in each format.

Excluding the special cases mentioned earlier, normal single-precision numbers span a range of $\pm 1.175494 \times 10^{-38}$ to $\pm 3.402824 \times 10^{38}$. They have a precision of about seven significant decimal digits (because $2^{-24} \approx 10^{-7}$). Similarly, normal double-precision numbers span a range of $\pm 2.22507385850720 \times 10^{-308}$ to $\pm 1.79769313486232 \times 10^{308}$ and have a precision of about 15 significant decimal digits. Quads have 34 decimal digits of precision but are not yet widely supported in hardware or software.

Rounding

Arithmetic results that fall outside of the available precision must round to a neighboring number. The rounding modes are round down, round up, round toward zero, and round to nearest. The default rounding mode is round to nearest. In the round-to-nearest mode, if two numbers are equally near, the one with a 0 in the least significant position of the fraction is chosen.

Recall that a number *overflows* when its magnitude is too large to be represented. Likewise, a number *underflows* when it is too tiny to be represented. In round-to-nearest mode, overflows are rounded up to $\pm\infty$ and underflows are rounded down to 0.

Floating-point cannot represent some numbers exactly, such as 1.7. However, when you type 1.7 into your calculator, you see exactly 1.7, not 1.69999.... To handle this, some applications, such as calculators and financial software, use *binary coded decimal* (BCD) numbers or formats with a base 10 exponent. BCD numbers encode each decimal digit using four bits with a range of 0 to 9. For example, the BCD fixed-point notation of 1.7 with four integer bits and four fraction bits would be 0001.0111. Of course, nothing is free. The cost is increased complexity in arithmetic hardware and wasted encodings (A–F encodings are not used) and, thus, decreased performance. So for compute-intensive applications, floating-point is much faster.

Floating-Point Addition

Addition with floating-point numbers is not as simple as addition with two’s complement numbers. The steps for adding floating-point numbers with the same sign are as follows:

- 1. Extract exponent and fraction bits.
- 2. Prepend leading 1 to form the mantissa.
- 3. Compare exponents.
- 4. Shift smaller mantissa if necessary.
- 5. Add mantissas.
- 6. Normalize mantissa and adjust exponent if necessary.
- 7. Round result.
- 8. Assemble exponent and fraction back into floating-point number.

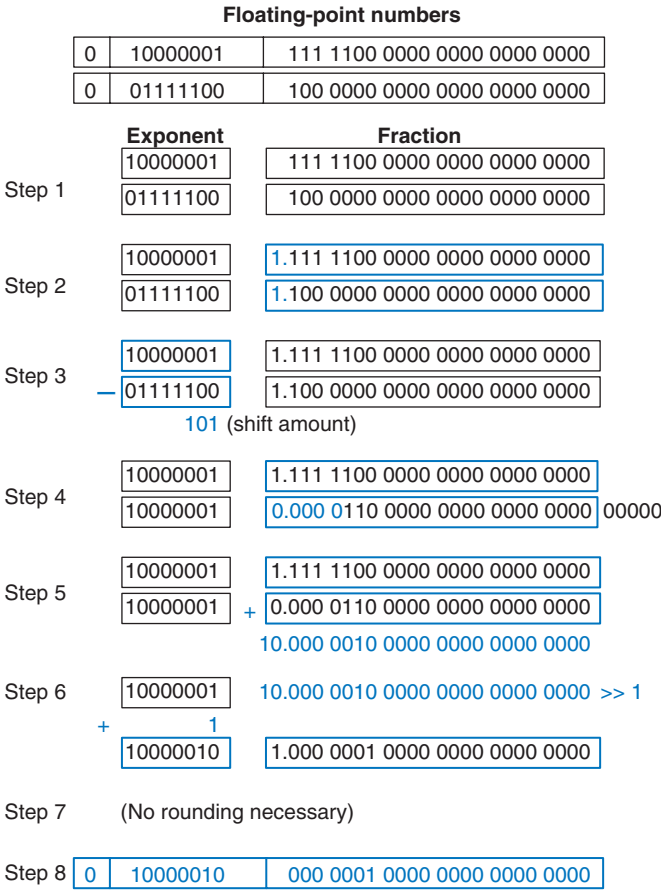


Figure 5.31 Floating-point addition

Figure 5.31 shows the floating-point addition of 7.875 (1.11111×2^2) and 0.1875 (1.1×2^{-3}). The result is 8.0625 (1.0000001×2^3). After the fraction and exponent bits are extracted and the implicit leading 1 is prepended in steps 1 and 2, the exponents are compared by subtracting the smaller exponent from the larger exponent. The result is the number of bits by which the smaller number is shifted to the right to align the implied binary point (i.e., to make the exponents equal) in step 4. The aligned numbers are added. Because the sum has a mantissa that is greater than or equal to 2.0, the result is normalized by shifting it to the right one bit and incrementing the exponent. In this example, the result is exact, so no rounding is necessary. The result is stored in floating-point notation by removing the implicit leading one of the mantissa and prepending the sign bit.

Floating-point arithmetic is usually done in hardware to make it fast. This hardware, called the *floating-point unit* (FPU), is typically distinct from the *central processing unit* (CPU). The infamous *floating-point division* (FDIV) bug in the Pentium FPU cost Intel \$475 million to recall and replace defective chips. The bug occurred simply because a lookup table was not loaded correctly.

5.4 SEQUENTIAL BUILDING BLOCKS

This section examines sequential building blocks, including counters and shift registers.

5.4.1 Counters

An N -bit *binary counter*, shown in Figure 5.32, is a sequential arithmetic circuit with clock and reset inputs and an N -bit output Q . *Reset* initializes the output to 0. The counter then advances through all 2^N possible outputs in binary order, incrementing on the rising edge of the clock.

Figure 5.33 shows an N -bit counter composed of an adder and a resettable register. On each cycle, the counter adds 1 to the value stored in the register. HDL Example 5.4 shows a binary counter with asynchronous reset, and Figure 5.34 shows the resulting hardware.

The most significant bit of an N -bit counter toggles every 2^N cycles. Thus, it reduces the frequency of the clock by a factor of 2^N . This is called a *divide-by- 2^N* counter and is useful for slowing down fast signals. For example, if a digital system has a 50 MHz internal clock, you can use a 24-bit counter to produce a $(50 \times 10^6 \text{ Hz} / 2^{24}) = 2.98 \text{ Hz}$ signal that blinks a light-emitting diode (LED) at a rate the human eye can observe.

A further counter generalization to produce arbitrary frequencies is called a *digitally controlled oscillator* (DCO, Example 5.7). Consider an N -bit counter that adds p on each cycle, rather than 1. If the counter receives a clock with frequency f_{clk} , the most significant bit now toggles at $f_{out} = f_{clk} \times p / 2^N$. With a judicious choice of p and N , you can produce an output of any frequency. Larger N gives more precise control at the expense of more hardware.

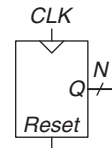


Figure 5.32 Counter symbol

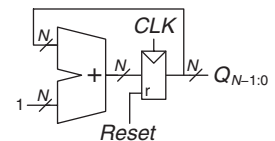


Figure 5.33 N -bit counter

HDL Example 5.4 COUNTER

SystemVerilog

```
module counter #(parameter N = 8)
    (input logic clk,
     input logic reset,
     output logic [N-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= q + 1;
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity counter is
    generic(N: integer := 8);
    port(clk, reset: in STD_LOGIC;
         q: out STD_LOGIC_VECTOR(N-1 downto 0));
end;

architecture synth of counter is
begin
    process(clk, reset) begin
        if reset then q <= (OTHERS => '0');
        elsif rising_edge(clk) then q <= q + '1';
        end if;
    end process;
end;
```

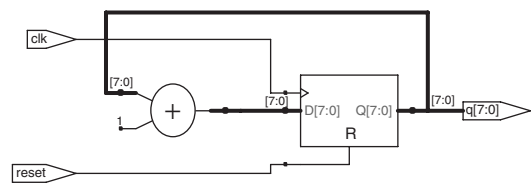


Figure 5.34 Synthesized counter

Example 5.7 DIGITALLY CONTROLLED OSCILLATOR

Suppose you have a 50 MHz clock and want to produce a 500 Hz output. Consider using an $N = 24$ - or 32-bit counter. What value of p should you choose and how close can you come to 500 Hz?

Solution We want $p/2^N = 500 \text{ Hz}/50 \text{ MHz} = 0.001$. If $N = 24$, choose $p = 168$ to get $f_{\text{out}} = 500.68 \text{ Hz}$. If $N = 32$, choose $p = 42950$ to get $f_{\text{out}} = 500.038 \text{ Hz}$.

Other types of counters, such as Up/Down counters, are explored in Exercises 5.51 through 5.54.

5.4.2 Shift Registers

A *shift register* has a clock, a serial input S_{in} , a serial output S_{out} , and N parallel outputs $Q_{N-1:0}$, as shown in Figure 5.35. On each rising edge of the clock, a new bit is shifted in from S_{in} and all the subsequent contents are shifted forward. The last bit in the shift register is available at S_{out} .

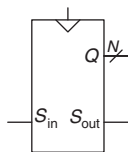


Figure 5.35 Shift register symbol

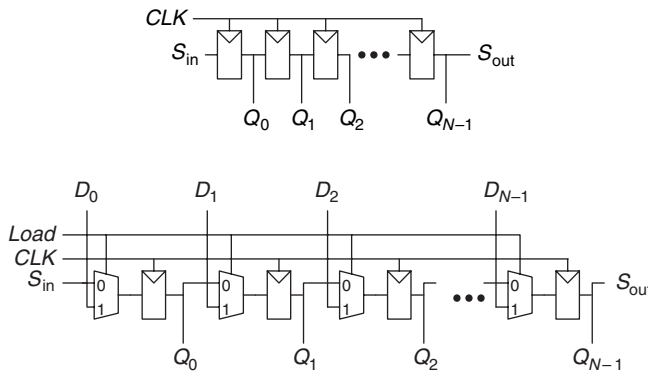


Figure 5.36 Shift register schematic

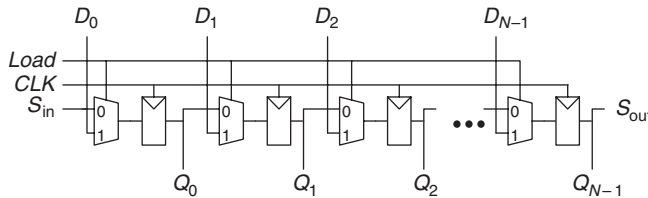


Figure 5.37 Shift register with parallel load

Shift registers can be viewed as *serial-to-parallel converters*. The input is provided serially (one bit at a time) at S_{in} . After N cycles, the past N inputs are available in parallel at Q .

A shift register can be constructed from N flip-flops connected in series, as shown in [Figure 5.36](#). Some shift registers also have a reset signal to initialize all of the flip-flops.

A related circuit is a *parallel-to-serial* converter that loads N bits in parallel, then shifts them out one at a time. A shift register can be modified to perform both serial-to-parallel and parallel-to-serial operations by adding a parallel input $D_{N-1:0}$ and a control signal *Load*, as shown in [Figure 5.37](#). When *Load* is asserted, the flip-flops are loaded in parallel from the D inputs. Otherwise, the shift register shifts normally. [HDL Example 5.5](#) describes such a shift register, and [Figure 5.38](#) shows the resulting hardware.

Scan Chains*

Shift registers are often used to test sequential circuits, using a technique called *scan chains*. Testing combinational circuits is relatively straightforward. Known inputs called *test vectors* are applied, and the outputs are checked against the expected result. Testing sequential circuits is more difficult because the circuits have state. Starting from a known initial condition, a large number of cycles of test vectors may be needed to put the circuit into a desired state. For example, testing that the most significant bit of a 32-bit counter advances from 0 to 1 requires resetting the counter, then applying 2^{31} (about two billion) clock pulses!

To solve this problem, designers like to be able to directly observe and control all of the machine's state. This is done by adding a test mode in which the contents of all flip-flops can be read out or loaded with desired values. Most systems have too many flip-flops to dedicate individual pins to read and write each flip-flop. Instead, all flip-flops in the system are connected together into a shift register called a *scan chain*. In

Don't confuse *shift registers* with the *shifters* from [Section 5.2.5](#). Shift registers are sequential logic blocks that shift in a new bit on each clock edge. Shifters are unclocked combinational logic blocks that shift an input by a specified amount.

HDL Example 5.5 SHIFT REGISTER WITH PARALLEL LOAD

SystemVerilog

```
module shiftreg #(parameter N = 8)
    (input logic      clk,
     input logic      reset, load,
     input logic      sin,
     input logic [N-1:0] d,
     output logic [N-1:0] q,
     output logic      sout);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else if (load) q <= d;
        else q <= {q[N-2:0], sin};

    assign sout = q[N-1];
endmodule
```

VHDL

```
library IEEE; use IEEE.STD_LOGIC_1164.ALL;

entity shiftreg is
    generic(N: integer := 8);
    port(clk, reset: in STD_LOGIC;
         load, sin: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(N-1 downto 0);
         q: out STD_LOGIC_VECTOR(N-1 downto 0);
         sout: out STD_LOGIC);
end;

architecture synth of shiftreg is
begin
    process(clk, reset) begin
        if reset = '1' then q <= (OTHERS => '0');
        elsif rising_edge(clk) then
            if load then q <= d;
            else q <= q(N-2 downto 0) & sin;
            end if;
        end if;
    end process;

    sout <= q(N-1);
end;
```

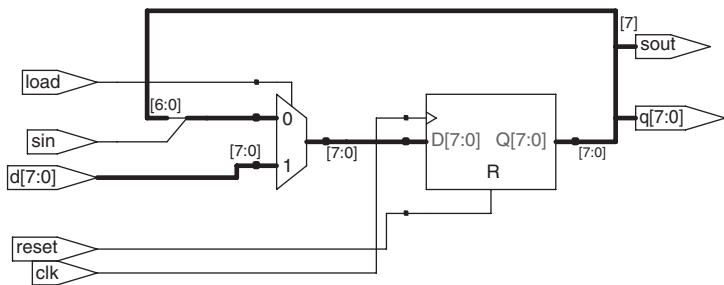


Figure 5.38 Synthesized shiftreg

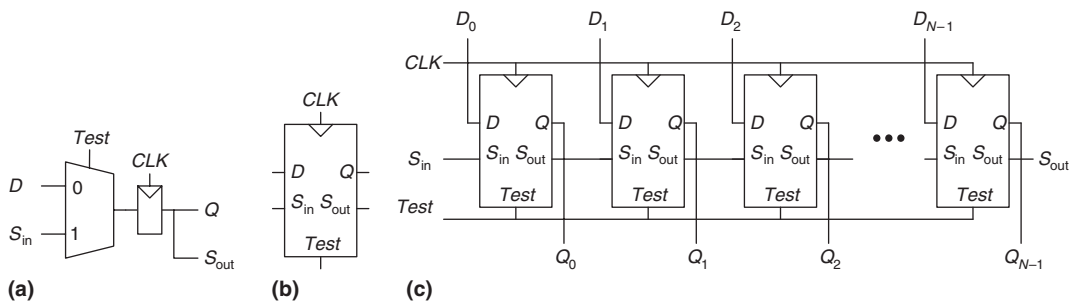


Figure 5.39 Scannable flip-flop: (a) schematic, (b) symbol, and (c) *N*-bit scannable register

normal operation, the flip-flops load data from their D input and ignore the scan chain. In test mode, the flip-flops serially shift their contents out and shift in new contents using S_{in} and S_{out} . The load multiplexer is usually integrated into the flip-flop to produce a *scannable flip-flop*. Figure 5.39 shows the schematic and symbol for a scannable flip-flop and illustrates how the flops are cascaded to build an N -bit scannable register.

For example, the 32-bit counter could be tested by shifting in the pattern 011111...111 in test mode, counting for one cycle in normal mode, then shifting out the result, which should be 100000...000. This requires only $32 + 1 + 32 = 65$ cycles.

5.5 MEMORY ARRAYS

The previous sections introduced arithmetic and sequential circuits for manipulating data. Digital systems also require *memories* to store the data used and generated by such circuits. Registers built from flip-flops are a kind of memory that stores small amounts of data. This section describes *memory arrays* that can efficiently store large amounts of data.

We begin with an overview describing characteristics shared by all memory arrays and then introduce three types of memory arrays: dynamic random access memory (DRAM), static random access memory (SRAM), and read only memory (ROM). Each memory differs in the way it stores data. We briefly discuss area and delay trade-offs and show how memory arrays are used, not only to store data but also to perform logic functions. We then finish this section by showing the HDL for a memory array.

5.5.1 Overview

Figure 5.40 shows a generic symbol for a memory array. The memory is organized as a two-dimensional array of memory cells. The memory reads or writes the contents of one of the rows of the array. This row is specified by an *address*. The value read or written is called *data*. An array with N -bit addresses and M -bit data has 2^N rows and M columns. Each row of data is called a *word*. Thus, the array contains 2^N M -bit words.

Figure 5.41 shows a memory array with two address bits and three data bits. The two address bits specify one of the four rows (data words) in the array. Each data word is three bits wide. Figure 5.41(b) shows some possible contents of the memory array.

The *depth* of an array is the number of rows, and the *width* is the number of columns, also called the *word size*. The size of an array is given as *depth* $\times$ *width*. Figure 5.41 is a 4-word $\times$ 3-bit array, or simply 4×3 array. The symbol for a 1024-word $\times$ 32-bit array is shown in Figure 5.42. The total size of this array is 32 kilobits (Kb), also referred to as 32 kibibits (Kib).

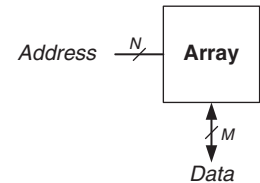


Figure 5.40 Generic memory array symbol

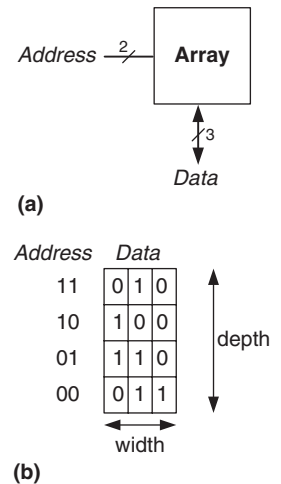


Figure 5.41 4×3 memory array: (a) symbol, (b) function

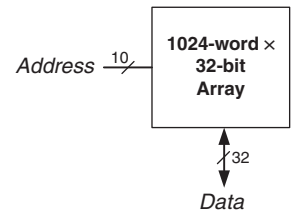


Figure 5.42 32-Kb array: depth = $2^{10} = 1024$ words, width = 32 bits

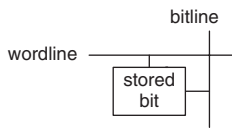


Figure 5.43 Bit cell

Bit Cells

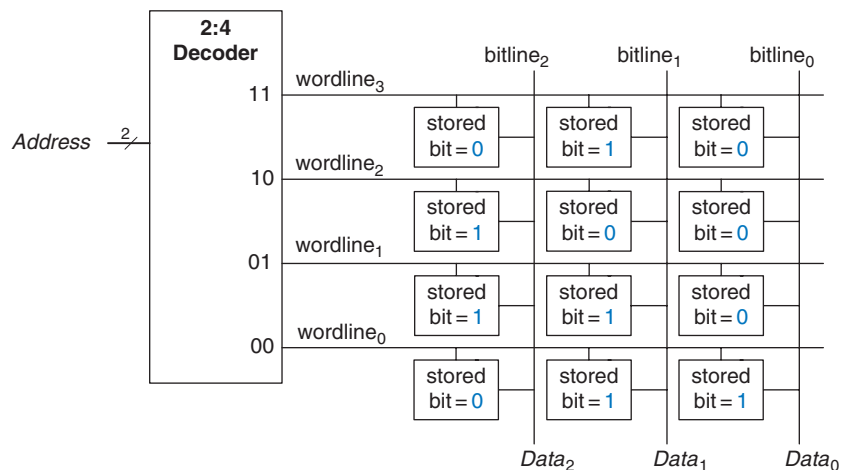
Memory arrays are built as an array of *bit cells*, each of which stores 1 bit of data. Figure 5.43 shows that each bit cell is connected to a *wordline* and a *bitline*. For each combination of address bits, the memory asserts a single wordline that activates the bit cells in that row. When the wordline is HIGH, the stored bit transfers to or from the bitline. Otherwise, the bitline is disconnected from the bit cell. The circuitry to store the bit varies with memory type.

To read a bit cell, the bitline is initially left floating (Z). Then, the wordline is turned ON, allowing the stored value to drive the bitline to 0 or 1. To write a bit cell, the bitline is strongly driven to the desired value. Then, the wordline is turned ON, connecting the bitline to the stored bit. The strongly driven bitline overpowers the contents of the bit cell, writing the desired value into the stored bit.

Organization

Figure 5.44 shows the internal organization of a 4×3 memory array. Of course, practical memories are much larger, but the behavior of larger arrays can be extrapolated from the smaller array. In this example, the array stores the data from Figure 5.41(b).

During a memory read, a wordline is asserted, and the corresponding row of bit cells drives the bitlines HIGH or LOW. During a memory write, the bitlines are driven HIGH or LOW first and then a wordline is asserted, allowing the bitline values to be stored in that row of bit cells. For example, to read *Address* 10, the bitlines are left floating, the decoder asserts *wordline*₂, and the data stored in that row of bit cells (100) reads out onto the *Data* bitlines. To write the value 001 to

Figure 5.44 4×3 memory array

Address 11, the bitlines are driven to the value 001, then *wordline*₃ is asserted and the new value (001) is stored in the bit cells.

Memory Ports

All memories have one or more *ports*. Each port gives read and/or write access to one memory address. The previous examples were all single-ported memories.

Multiported memories can access several addresses simultaneously. Figure 5.45 shows a three-ported memory with two read ports and one write port. Port 1 reads the data from address *A1* onto the read data output *RD1*. Port 2 reads the data from address *A2* onto *RD2*. Port 3 writes the data from the write data input *WD3* into address *A3* on the rising edge of the clock if the write enable *WE3* is asserted.

Memory Types

Memory arrays are specified by their size (depth × width) and the number and type of ports. All memory arrays store data as an array of bit cells, but they differ in how they store bits.

Memories are classified based on how they store bits in the bit cell. The broadest classification is *random access memory* (RAM) versus *read only memory* (ROM). RAM is *volatile*, meaning that it loses its data when the power is turned off. ROM is *nonvolatile*, meaning that it retains its data indefinitely, even without a power source.

RAM and ROM received their names for historical reasons that are no longer very meaningful. RAM is called *random* access memory because any data word is accessed with the same delay as any other. In contrast, a *sequential* access memory, such as a tape recorder, accesses nearby data more quickly than faraway data (e.g., at the other end of the tape). ROM is called *read only* memory because, historically, it could only be read but not written. These names are confusing, because ROMs are also randomly accessed. Worse yet, most modern ROMs can be written as well as read! The important distinction to remember is that RAMs are volatile and ROMs are nonvolatile.

The two major types of RAMs are *dynamic RAM* (DRAM) and *static RAM* (SRAM). Dynamic RAM stores data as a charge on a capacitor, whereas static RAM stores data using a pair of cross-coupled inverters. There are many flavors of ROMs that vary by how they are written and erased. These various types of memories are discussed in the subsequent sections.

5.5.2 Dynamic Random Access Memory (DRAM)

DRAM, pronounced “dee-ram,” stores a bit as the presence or absence of charge on a capacitor. Figure 5.46 shows a DRAM bit cell. The bit value is stored on a capacitor. The nMOS transistor behaves as a switch

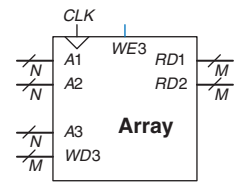


Figure 5.45 Three-ported memory



Robert Dennard, 1932–

Invented DRAM in 1966 at IBM. Although many were skeptical that the idea would work, by the mid-1970's DRAM was in virtually all computers. He claims to have done little creative work until, arriving at IBM, they handed him a patent notebook and said, “put all your ideas in there.” Since 1965, he has received 35 patents in semiconductors and microelectronics. (Photo courtesy of IBM.)

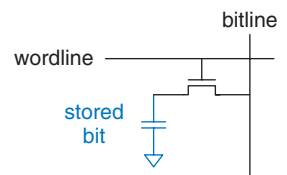
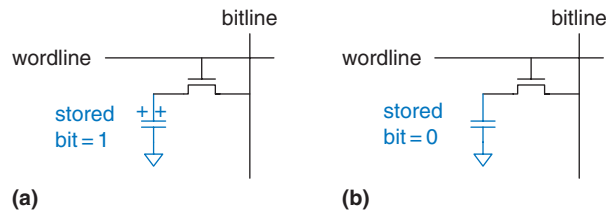


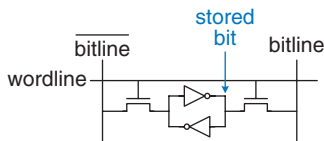
Figure 5.46 DRAM bit cell

Figure 5.47 DRAM stored values

that either connects or disconnects the capacitor from the bitline. When the wordline is asserted, the nMOS transistor turns ON, and the stored bit value transfers to or from the bitline.

As shown in Figure 5.47(a), when the capacitor is charged to V_{DD} , the stored bit is 1; when it is discharged to GND (Figure 5.47(b)), the stored bit is 0. The capacitor node is *dynamic* because it is not actively driven HIGH or LOW by a transistor tied to V_{DD} or GND.

Upon a read, data values are transferred from the capacitor to the bitline. Upon a write, data values are transferred from the bitline to the capacitor. Reading destroys the bit value stored on the capacitor, so the data word must be restored (rewritten) after each read. Even when DRAM is not read, the contents must be refreshed (read and rewritten) every few milliseconds, because the charge on the capacitor gradually leaks away.

**Figure 5.48** SRAM bit cell

5.5.3 Static Random Access Memory (SRAM)

SRAM, pronounced “es-ram,” is *static* because stored bits do not need to be refreshed. Figure 5.48 shows an SRAM bit cell. The data bit is stored on cross-coupled inverters like those described in Section 3.2. Each cell has two outputs, bitline and $\bar{\text{bitline}}$. When the wordline is asserted, both nMOS transistors turn on, and data values are transferred to or from the bitlines. Unlike DRAM, if noise degrades the value of the stored bit, the cross-coupled inverters restore the value.

5.5.4 Area and Delay

Flip-flops, SRAMs, and DRAMs are all volatile memories, but each has different area and delay characteristics. Table 5.6 shows a comparison of these three types of volatile memory. The data bit stored in a flip-flop is available immediately at its output. But flip-flops take at least 20 transistors to build. Generally, the more transistors a device has, the more area, power, and cost it requires. DRAM latency is longer than that of SRAM because its bitline is not actively driven by a transistor. DRAM must wait for charge to move (relatively) slowly from the capacitor to the bitline. DRAM also fundamentally has lower throughput

Table 5.6 Memory comparison

Memory Type	Transistors per Bit Cell	Latency
Flip-flop	~20	Fast
SRAM	6	Medium
DRAM	1	Slow

than SRAM, because it must refresh data periodically and after a read. DRAM technologies such as *synchronous DRAM* (SDRAM) and *double data rate* (DDR) SDRAM have been developed to overcome this problem. SDRAM uses a clock to pipeline memory accesses. DDR SDRAM, sometimes called simply DDR, uses both the rising and falling edges of the clock to access data, thus doubling the throughput for a given clock speed. DDR was first standardized in 2000 and ran at 100 to 200 MHz. Later standards, DDR2, DDR3, and DDR4, increased the clock speeds, with speeds in 2021 being over 3 GHz.

Memory latency and throughput also depend on memory size; larger memories tend to be slower than smaller ones if all else is the same. The best memory type for a particular design depends on the speed, cost, and power constraints.

5.5.5 Register Files

Digital systems often use a number of registers to store temporary variables. This group of registers, called a *register file*, is usually built as a small, multiported SRAM array because it is more compact than an array of flip-flops. In some register files, a particular entry, such as register 0, is hardwired to always read the value 0 because 0 is a commonly used constant.

Figure 5.49 shows a 32-register $\times$ 32-bit three-ported register file built from the three-ported memory of Figure 5.45. The register file has two read ports (A1/RD1 and A2/RD2) and one write port (A3/WD3). The 5-bit addresses—A1, A2, and A3—can each access all $2^5 = 32$ registers. So, two registers can be read and one register written simultaneously.

5.5.6 Read Only Memory (ROM)

ROM stores a bit as the presence or absence of a transistor. Figure 5.50 shows a simple ROM bit cell. To read the cell, the bitline is weakly pulled HIGH. Then, the wordline is turned ON. If the transistor is present, it pulls the bitline LOW. If it is absent, the bitline remains HIGH.

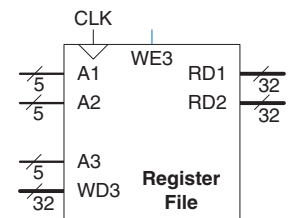


Figure 5.49 32 $\times$ 32 register file with two read ports and one write port

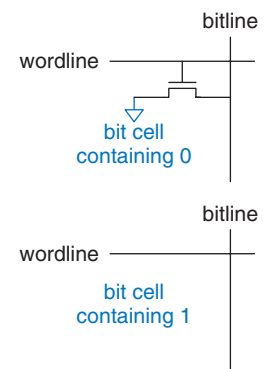


Figure 5.50 ROM bit cells containing 0 and 1

Note that the ROM bit cell is a combinational circuit and has no state to “forget” if power is turned off.

The contents of a ROM can be indicated using *dot notation*. Figure 5.51 shows the dot notation for a 4-word $\times$ 3-bit ROM containing the data from Figure 5.41. A dot at the intersection of a row (wordline) and a column (bitline) indicates that the data bit is 1. For example, the top wordline has a single dot on $Data_1$, so the data word stored at Address 11 is 010.

Conceptually, ROMs can be built using two-level logic with a group of AND gates followed by a group of OR gates. The AND gates produce all possible minterms and, hence, form a decoder. Figure 5.52 shows the ROM of Figure 5.51 built using a decoder and OR gates. Each dotted row in Figure 5.51 is an input to an OR gate in Figure 5.52. For

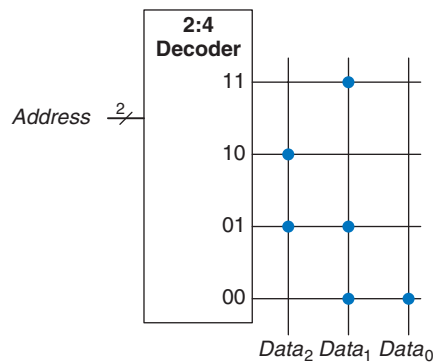


Figure 5.51 4 $\times$ 3 ROM: dot notation

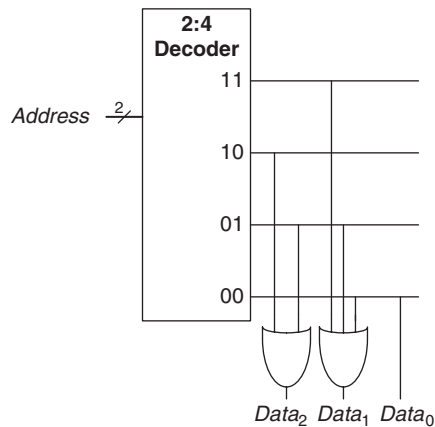


Figure 5.52 4 $\times$ 3 ROM implementation using gates

data bits with a single dot—in this case, $Data_0$ —no OR gate is needed. This representation of a ROM is interesting because it shows how the ROM can perform any two-level logic function. In practice, ROMs are built from transistors instead of logic gates to reduce their size and cost. Section 5.6.3 explores the transistor-level implementation further.

The contents of the ROM bit cell in Figure 5.50 are specified during manufacturing by the presence or absence of a transistor in each bit cell. A *programmable ROM* (PROM, pronounced like the dance) places a transistor in every bit cell but provides a way to connect or disconnect the transistor to ground.

Figure 5.53 shows the bit cell for a *fuse-programmable ROM*. The user programs the ROM by applying a high voltage to selectively blow fuses. If the fuse is present, the transistor is connected to GND and the cell holds a 0. If the fuse is destroyed, the transistor is disconnected from ground and the cell holds a 1. This is also called a *one-time programmable ROM*, because the fuse cannot be repaired once it is blown.

Reprogrammable ROMs provide a reversible mechanism for connecting or disconnecting the transistor to GND. *Erasable PROMs* (EPROMs, pronounced “e-proms”) replace the nMOS transistor and fuse with a *floating-gate transistor*. The floating gate is not physically attached to any other wires. When suitable high voltages are applied, electrons tunnel through an insulator onto the floating gate, turning on the transistor and connecting the bitline to the wordline (decoder output). When the EPROM is exposed to intense ultraviolet (UV) light for about half an hour, the electrons are knocked off the floating gate, turning the transistor off. These actions are called *programming* and *erasing*, respectively. *Electrically erasable PROMs* (EEPROMs, pronounced “e-e-proms” or “double-e proms”) and *Flash* memory use similar principles but include circuitry on the chip for erasing as well as programming, so no UV light is necessary. EEPROM bit cells are individually erasable; Flash memory erases larger blocks of bits and is cheaper because fewer erasing circuits are needed. In 2021, Flash memory cost about \$0.10 per GB, and the price continues to drop by 30% to 40% per year. Flash has become an extremely popular way to store large amounts of data in portable battery-powered systems such as cameras and music players.

In summary, modern ROMs are not really read only; they can be programmed (written) as well. The difference between RAM and ROM is that ROMs take a longer time to write but are nonvolatile.

5.5.7 Logic Using Memory Arrays

Although they are used primarily for data storage, memory arrays can also perform combinational logic functions. For example, the $Data_2$ output of the ROM in Figure 5.51 is the XOR of the two *Address* inputs. Likewise,

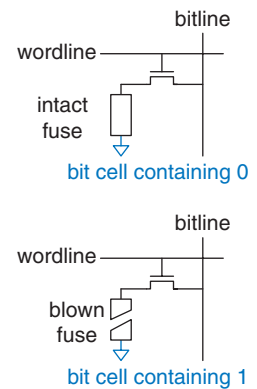


Figure 5.53 Fuse-programmable ROM bit cell

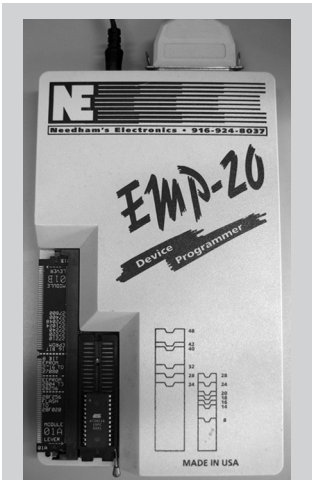


Fujio Masuoka, 1944–

Received a Ph.D. in electrical engineering from Tohoku University, Japan. Developed memories and high-speed circuits at Toshiba from 1971 to 1994. Invented Flash memory as an unauthorized project pursued during nights and weekends in the late 1970's. Flash received its name because the process of erasing the memory reminds one of the flash of a camera. Toshiba was slow to commercialize the idea; Intel was first to market in 1988. Flash has grown into a \$170 billion per year market. Dr. Masuoka later joined the faculty at Tohoku University and is working to develop a 3-dimensional transistor.



Flash memory drives with Universal Serial Bus (USB) connectors have replaced floppy disks and CDs for sharing files because Flash costs have dropped so dramatically.



Programmable ROMs can be configured with a device programmer like the one shown below. The device programmer is attached to a computer, which specifies the type of ROM and the data values to program. The device programmer blows fuses or injects charge onto a floating gate on the ROM. Thus, the programming process is sometimes called *burning* a ROM.

$Data_0$ is the NOR of the two inputs. A 2^N -word $\times$ M -bit memory can perform any combinational function of N inputs and M outputs. For example, the ROM in Figure 5.51 performs three functions of two inputs.

Memory arrays used to perform logic are called *lookup tables* (LUTs). Figure 5.54 shows a 4-word $\times$ 1-bit memory array used as a lookup table to perform the function $Y = AB$. Using memory to perform logic, the user can look up the output value for a given input combination (address). Each address corresponds to a row in the truth table, and each data bit corresponds to an output value.

5.5.8 Memory HDL

HDL Example 5.6 describes a 2^N -word $\times$ M -bit RAM, and Figure 5.55 shows the resulting hardware. The RAM has a synchronous enabled write. In other words, writes occur on the rising edge of the clock if the write enable *we* is asserted. Reads occur immediately. When power is first applied, the contents of the RAM are unpredictable.

HDL Example 5.7 describes a 4-word $\times$ 3-bit ROM. The contents of the ROM are specified in the HDL case statement. A ROM as small as this one may be synthesized into logic gates rather than an array. Note that the seven-segment decoder from HDL Example 4.24 synthesizes into a ROM in Figure 4.20. HDL Example 5.8 shows a 3-ported 32×32 register file with entry 0 hardwired to 0.

5.6 LOGIC ARRAYS

Like memory, gates can be organized into regular arrays. If the connections are made programmable, these *logic arrays* can be configured to

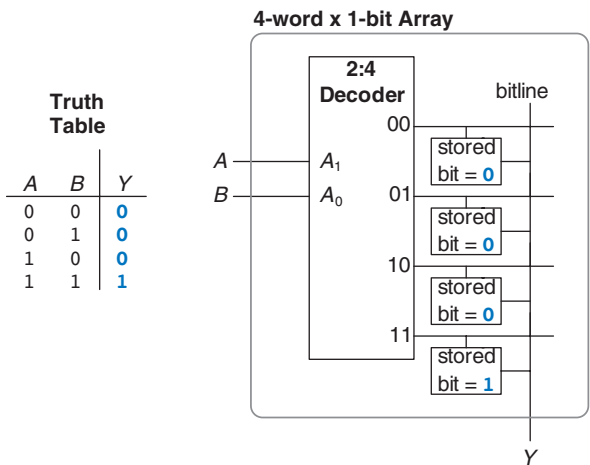


Figure 5.54 4-word $\times$ 1-bit memory array used as a lookup table

HDL Example 5.6 RAM**SystemVerilog**

```

module ram #(parameter N = 6, M = 32)
  (input  logic      clk,
   input  logic      we,
   input  logic [N-1:0] adr,
   input  logic [M-1:0] din,
   output logic [M-1:0] dout);

  logic [M-1:0] mem [2**N-1:0];

  always_ff @(posedge clk)
    if (we) mem [adr] <= din;

  assign dout = mem[adr];
endmodule

```

VHDL

```

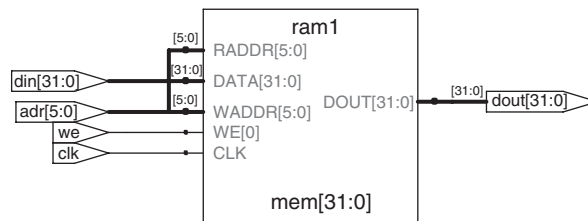
library IEEE; use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD_UNSIGNED.ALL;

entity ram_array is
  generic(N: integer := 6; M: integer := 32);
  port(clk,
        we: in STD_LOGIC;
        adr: in STD_LOGIC_VECTOR(N-1 downto 0);
        din: in STD_LOGIC_VECTOR(M-1 downto 0);
        dout: out STD_LOGIC_VECTOR(M-1 downto 0));
end;

architecture synth of ram_array is
  type mem_array is array ((2**N-1) downto 0)
    of STD_LOGIC_VECTOR (M-1 downto 0);
  signal mem: mem_array;
begin
  process(clk) begin
    if rising_edge(clk) then
      if we then mem(TO_INTEGER(adr)) <= din;
      end if;
    end if;
  end process;

  dout <= mem(TO_INTEGER(adr));
end;

```

**Figure 5.55** Synthesized ram

perform any function without the user having to connect wires in specific ways. The regular structure simplifies design. Logic arrays are mass-produced in large quantities, so they are inexpensive. Software tools allow users to map logic designs onto these arrays. Most logic arrays are also reconfigurable, allowing designs to be modified without replacing the hardware. Reconfigurability is valuable during development and is also useful in the field because a system can be upgraded by simply downloading the new configuration.

This section introduces two types of logic arrays: programmable logic arrays (PLAs), and field programmable gate arrays (FPGAs).

HDL Example 5.7 ROM**SystemVerilog**

```

module rom(input  logic [1:0] adr,
           output logic [2:0] dout);

    always_comb
    case(adr)
        2'b00: dout = 3'b011;
        2'b01: dout = 3'b110;
        2'b10: dout = 3'b100;
        2'b11: dout = 3'b010;
    endcase
endmodule

```

VHDL

```

library IEEE; use IEEE.STD_LOGIC_1164.all;

entity rom is
    port(adr: in  STD_LOGIC_VECTOR(1 downto 0);
         dout: out STD_LOGIC_VECTOR(2 downto 0));
end;

architecture synth of rom is
begin
    process(all) begin
        case adr is
            when "00" => dout <= "011";
            when "01" => dout <= "110";
            when "10" => dout <= "100";
            when "11" => dout <= "010";
            when others => dout <= "---";
        end case;
    end process;
end;

```

HDL Example 5.8 REGISTER FILE**SystemVerilog**

```

module regfile(input  logic      clk,
               input  logic      we3,
               input  logic [5:0] a1, a2, a3,
               input  logic [31:0] wd3,
               output logic [31:0] rd1, rd2);

    logic [31:0] rf[31:0];
    // three ported register file
    // read two ports combinationaly (A1/RD1, A2/RD2)
    // write third port on rising edge of clock (A3/WD3/WE3)
    // register 0 hardwired to 0

    always_ff @(posedge clk)
        if (we3) rf[a3] <= wd3;

    assign rd1 = (a1 != 0) ? rf[a1] : 0;
    assign rd2 = (a2 != 0) ? rf[a2] : 0;
endmodule

```

VHDL

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity regfile is
    port(clk:      in  STD_LOGIC;
         we3:      in  STD_LOGIC;
         a1, a2, a3: in  STD_LOGIC_VECTOR(5 downto 0);
         wd3:      in  STD_LOGIC_VECTOR(31 downto 0);
         rd1, rd2: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of regfile is
    type ramtype is array (31 downto 0) of STD_LOGIC_VECTOR
        (31 downto 0);

    signal mem: ramtype;

begin
    -- three ported register file
    -- read two ports combinationaly (A1/RD1, A2/RD2)
    -- write third port on rising edge of clock (A3/WD3/WE3)
    -- register 0 hardwired to 0
    process(clk) begin
        if rising_edge(clk) then
            if we3 = '1' then mem(to_integer(a3)) <= wd3;
            end if;
        end if;
    end process;

    process(a1, a2) begin
        if (to_integer(a1) = 0) then rd1 <= X"00000000";
        else rd1 <= mem(to_integer(a1));
        end if;
        if (to_integer(a2) = 0) then rd2 <= X"00000000";
        else rd2 <= mem(to_integer(a2));
        end if;
    end process;
end;

```

PLAs, the older technology, perform only combinational logic functions. FPGAs can perform both combinational and sequential logic.

5.6.1 Programmable Logic Array (PLA)

PLAs implement two-level combinational logic in sum-of-products (SOP) form. PLAs are built from an AND array followed by an OR array, as shown in Figure 5.56. The inputs (in true and complementary form) drive an AND array, which produces implicants. The implicants, in turn, are ORed together to form the outputs. An $M \times N \times P$ -bit PLA has M inputs, N implicants, and P outputs.

Figure 5.57 shows the dot notation for a $3 \times 3 \times 2$ -bit PLA performing the functions $X = \bar{A}\bar{B}C + AB\bar{C}$ and $Y = A\bar{B}$. Each row in the AND array forms an implicant. Dots in each row of the AND array indicate which literals comprise the implicant. The AND array in Figure 5.57 forms three implicants: $\bar{A}\bar{B}C$, $AB\bar{C}$, and $A\bar{B}$. Dots in the OR array indicate which implicants are part of the output function.

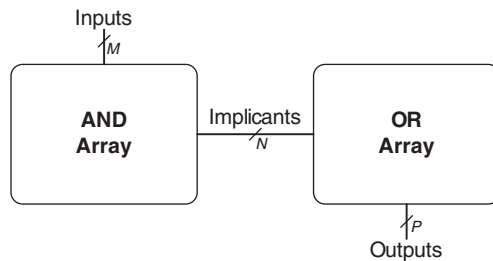


Figure 5.56 $M \times N \times P$ -bit PLA

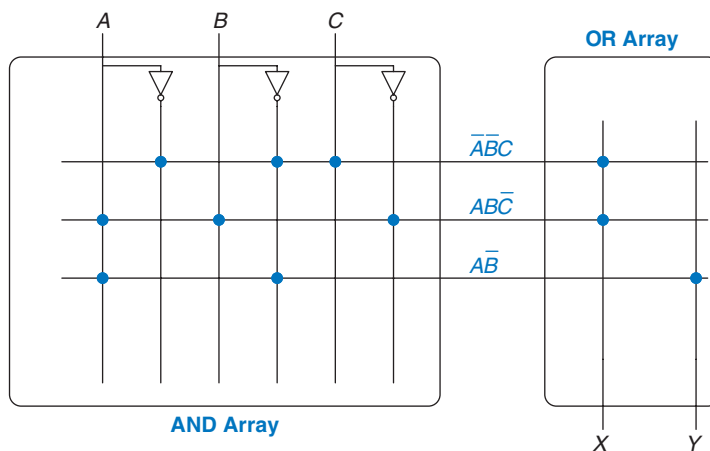


Figure 5.57 $3 \times 3 \times 2$ -bit PLA: dot notation

Figure 5.58 $3 \times 3 \times 2$ -bit PLA using two-level logic

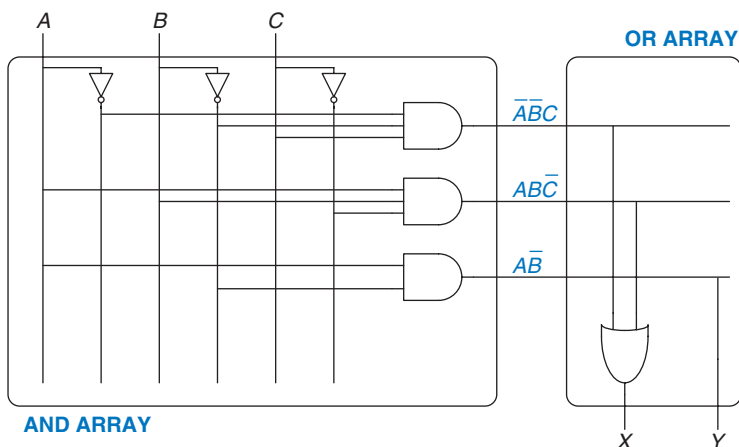


Figure 5.58 shows how PLAs can be built using two-level logic. An alternative implementation is given in Section 5.6.3.

ROMs can be viewed as a special case of PLAs. A 2^M -word $\times N$ -bit ROM is simply an $M \times 2^M \times N$ -bit PLA. The decoder behaves as an AND plane that produces all 2^M minterms. The ROM array behaves as an OR plane that produces the outputs. If the function does not depend on all 2^M minterms, a PLA is likely to be smaller than a ROM. For example, an 8-word $\times$ 2-bit ROM is required to perform the same functions performed by the $3 \times 3 \times 2$ -bit PLA shown in Figures 5.57 and 5.58.

Simple programmable logic devices (SPLDs) are souped-up PLAs that add registers and various other features to the basic AND/OR planes. However, SPLDs and PLAs have largely been displaced by FPGAs, which are more flexible and efficient for building large systems.

5.6.2 Field Programmable Gate Array (FPGA)

An FPGA is an array of reconfigurable gates. Using software programming tools, a user can implement designs on the FPGA employing either an HDL or a schematic. FPGAs are more powerful and more flexible than PLAs for several reasons. They can implement both combinational and sequential logic. They can also implement multilevel logic functions, whereas PLAs can implement only two-level logic. Modern FPGAs integrate other useful features, such as built-in multipliers, high-speed I/Os, data converters including analog-to-digital converters, large RAM arrays, and processors.

FPGAs are built as an array of configurable *logic elements* (LEs), also referred to as *configurable logic blocks* (CLBs). Each LE can be configured to perform combinational or sequential functions. Figure 5.59



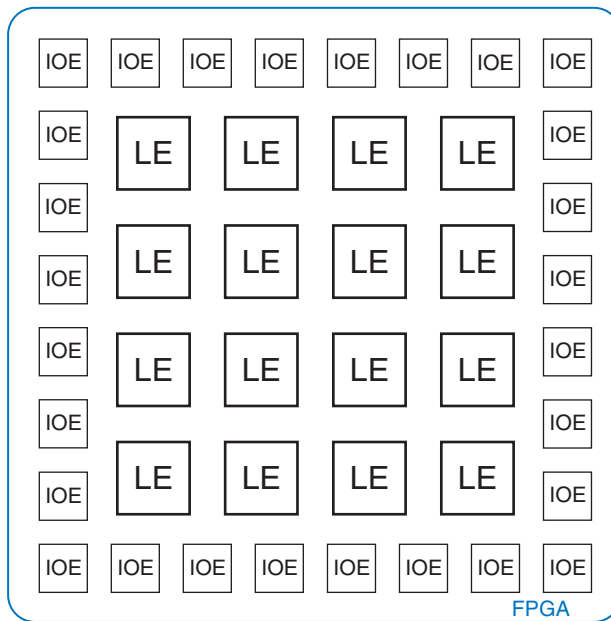


Figure 5.59 General FPGA layout

FPGAs are the brains of many consumer products, including automobiles, medical equipment, and media devices, such as MP3 players. The Mercedes Benz S-Class series, for example, has over a dozen Xilinx FPGAs or PLDs for uses ranging from entertainment to navigation to cruise control systems. FPGAs allow for quick time to market and make debugging or adding features late in the design process easier.

shows a general block diagram of an FPGA. The LEs are surrounded by *input/output elements* (IOEs) for interfacing with the outside world. The IOEs connect LE inputs and outputs to pins on the chip package. LEs can connect to other LEs and IOEs through programmable routing channels.

Two of the leading FPGA manufacturers are Intel (formerly, Altera Corp.) and Xilinx, Inc. [Figure 5.60](#) shows a single LE from Intel's Cyclone IV FPGA, introduced in 2009. The key elements of the LE are a 4-input lookup table (LUT) and a 1-bit register. The LE also contains configurable multiplexers to route signals through the LE. The FPGA is configured by specifying the contents of the LUTs and the select signals for the multiplexers.

Each Cyclone IV LE has one 4-input LUT and one flip-flop. By loading the appropriate values into the LUT, it can be configured to perform any function of up to four variables. Configuring the FPGA also involves choosing the select signals that determine how the multiplexers route data through the LE and to neighboring LEs and IOEs. For example, depending on the multiplexer configuration, the LUT may receive one of its inputs from either *data 3* or the output of the LE's own register. The other three inputs always come from *data 1*, *data 2*, and *data 4*. The *data 1–4* inputs come from IOEs or the outputs of other LEs, depending on routing external to the LE. The LUT output either goes directly to

Altera Corporation was founded in 1983 as one of the original FPGA manufacturers, along with Xilinx Inc., which was founded in 1984. Altera was acquired by Intel in 2015 for \$16.7 billion.

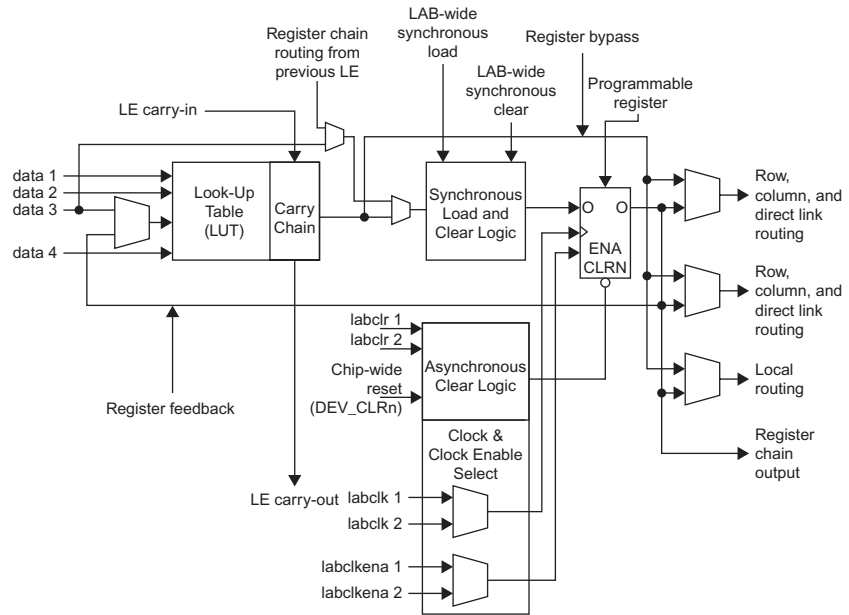


Figure 5.60 Cyclone IV Logic Element (LE)

(Reproduced with permission from the Altera Cyclone™ IV Handbook © 2010 Altera Corporation)

the LE output for combinational functions, or it can be fed through the flip-flop for registered functions. The flip-flop input comes from its own LUT output, the *data 3* input, or the register output of the previous LE. Additional hardware includes support for addition using the carry chain hardware, other multiplexers for routing, and flip-flop enable and reset. Altera groups 16 LEs together to create a *logic array block (LAB)* and provides local connections between LEs within the LAB.

In summary, each Cyclone IV LE can perform one combinational and/or registered function, which can involve up to four variables. Other brands of FPGAs are organized somewhat differently, but the same general principles apply. For example, Xilinx's 7-series FPGAs use 6-input LUTs instead of 4-input LUTs.

The designer configures an FPGA by first creating a schematic or HDL description of the design. The design is then synthesized onto the FPGA. The synthesis tool determines how the LUTs, multiplexers, and routing channels should be configured to perform the specified functions. This configuration information is then downloaded to the FPGA. Because Cyclone IV FPGAs store their configuration information in SRAM, they are easily reprogrammed. The FPGA may download its SRAM contents from a computer in the laboratory or from

an EEPROM chip when the system is turned on. Some manufacturers include an EEPROM directly on the FPGA or use one-time programmable fuses to configure the FPGA.

Example 5.8 FUNCTIONS BUILT USING LEs

Explain how to configure one or more Cyclone IV LEs to perform the following functions: (a) $X = \overline{A}BC + AB\overline{C}$ and $Y = A\overline{B}$ (b) $Y = JKLM\overline{PQR}$; (c) a divide-by-3 counter with binary state encoding (see Figure 3.29(a)). You may show interconnection between LEs as needed.

Solution (a) Configure two LEs. One LUT computes X and the other LUT computes Y , as shown in Figure 5.61. For the first LE, inputs *data 1*, *data 2*, and *data 3* are A , B , and C , respectively (these connections are set by the routing channels). *data 4* is a don't care but must be tied to something, so it is tied to 0. For the second LE, inputs *data 1* and *data 2* are A and B , respectively; the other LUT inputs are don't cares and are tied to 0. Configure the final multiplexers to select the combinational outputs from the LUTs to produce X and Y . In general, a single LE can compute any function of up to four input variables in this fashion.

(b) Configure the LUT of the first LE to compute $X = JKLM$ and the LUT on the second LE to compute $Y = X\overline{PQR}$. Configure the final multiplexers to select the combinational outputs X and Y from each LE. This configuration is shown in Figure 5.62. Routing channels between LEs, indicated by the dashed blue lines, connect the output of LE 1 to the input of LE 2. In general, a group of LEs can compute functions of N input variables in this manner.

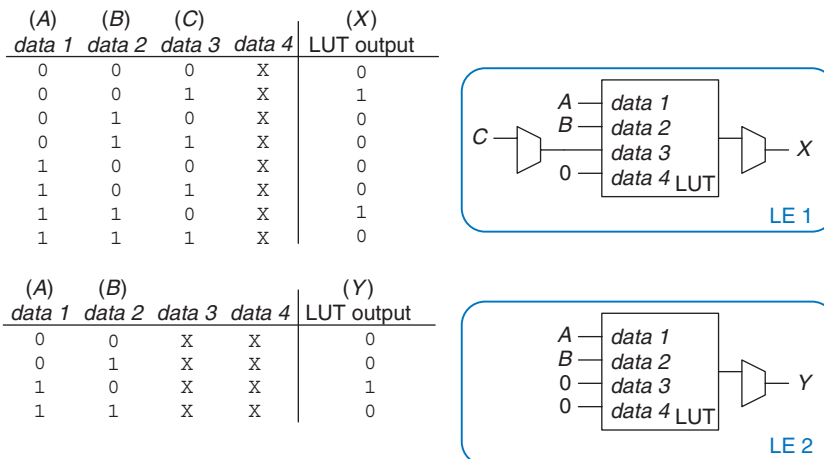


Figure 5.61 LE configuration for two functions of up to four inputs each

(c) The FSM has two bits of state ($S_{1:0}$) and one output (Y). The next state depends on the two bits of current state. Use two LEs to compute the next state from the current state, as shown in Figure 5.63. Use the two flip-flops, one from each LE, to hold this state. The flip-flops have a reset input that can be connected to an external *Reset* signal. The registered outputs are fed back to the LUT inputs using the multiplexer on *data 3* and routing channels between LEs, as indicated by the dashed blue lines. In general, another LE might be necessary to compute the output Y . However, in this case, $Y = S'_0$, so Y can come from LE 1. Hence, the entire FSM fits in two LEs. In general, an FSM requires at least one LE for each bit of state, and it may require more LEs for the output or next state logic if they are too complex to fit in a single LUT.

Figure 5.62 LE configuration for one function of more than four inputs

(J)	(K)	(L)	(M)	(X)	(P)	(Q)	(R)	(X)	(Y)
data 1	data 2	data 3	data 4	LUT output	data 1	data 2	data 3	data 4	LUT output
0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	1	0
0	0	1	0	0	0	0	1	0	0
0	0	1	1	0	0	0	1	1	0
0	1	0	0	0	0	1	0	0	0
0	1	0	1	0	0	1	0	1	0
0	1	1	0	0	0	1	1	0	0
0	1	1	1	0	0	1	1	1	0
1	0	0	0	0	1	0	0	0	0
1	0	0	1	0	1	0	0	1	0
1	0	1	0	0	1	0	1	0	0
1	0	1	1	0	1	0	1	1	0
1	1	0	0	0	1	1	0	0	0
1	1	0	1	0	1	1	0	1	0
1	1	1	0	0	1	1	1	0	0
1	1	1	1	1	1	1	1	1	1

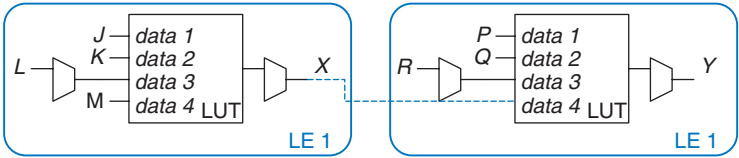
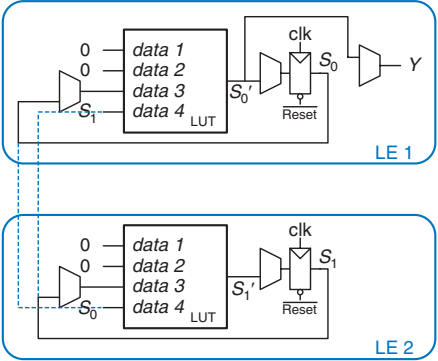


Figure 5.63 LE configuration for FSM with two bits of state

data 1	data 2	(S_0)	(S_1)	(S'_0)
data 1	data 2	data 3	data 4	LUT output
X	X	0	0	1
X	X	0	1	0
X	X	1	0	0
X	X	1	1	0

data 1	data 2	(S_1)	(S_0)	(S'_1)
data 1	data 2	data 3	data 4	LUT output
X	X	0	0	0
X	X	0	1	1
X	X	1	0	0
X	X	1	1	0



Example 5.9 MORE LOGIC ELEMENT EXAMPLES

How many Cyclone IV LEs are required to build each of the following circuits?

- (a) 4-input AND
- (b) 7-input XOR
- (c) $Y = A(B + C + D + E) + \overline{A}(BCDE)$
- (d) 12-bit shift register
- (e) 32-bit 2:1 multiplexer
- (f) 16-bit counter
- (g) Arbitrary finite state machine with 2 bits of state, 2 inputs, and 3 outputs

Solution

- (a) 1: The LUT can perform any function of up to 4 inputs.
- (b) 2: The first LUT can compute a 4-input XOR. The second LUT can XOR that output with three more inputs.
- (c) 3: One LUT computes $B + C + D + E$, a function of 4 inputs. A second LUT computes $BCDE$, a different function of 4 inputs. A third LUT uses 3 inputs (these two outputs and A) to compute Y .
- (d) 12: A shift register contains one flip-flop per bit.
- (e) 32: A 2:1 multiplexer is a function of three inputs: S , D_0 , and D_1 , so it requires one LUT per bit.
- (f) 16: Each bit of a counter requires a flip-flop and a full adder. The LE contains the flip-flop and adder logic. Although a full adder has two outputs and might seem to need two LUTs, the LE contains special carry chain logic shown in [Figure 5.60](#) optimized to perform addition with a single LE.
- (g) 5: The FSM has two flip-flops, two next state signals, and three output signals. Each next state signal is a function of four variables (the two bits of state and two inputs), so it can be computed with one LUT. Thus, two LEs are sufficient for the next state logic and state register. Each output is a function of at most four signals, so one more LUT is needed for each output.

Example 5.10 LE DELAY

Alyssa P. Hacker is building a finite state machine that must run at 200 MHz. She uses a Cyclone IV FPGA with the following specifications: $t_{LE} = 381$ ps per LE, $t_{\text{setup}} = 76$ ps, and $t_{pcq} = 199$ ps for all flip-flops. The wiring delay between LEs is 246 ps. Assume that the hold time for the flip-flops is 0. What

is the maximum number of levels of LEs that her design can use between two registers?

Solution Alyssa uses Equation 3.13 to solve for the maximum propagation delay of the logic: $t_{pd} \leq T_c - (t_{pcq} + t_{setup})$.

Thus, $t_{pd} = 5 \text{ ns} - (0.199 \text{ ns} + 0.076 \text{ ns})$, so $t_{pd} \leq 4.725 \text{ ns}$. The delay of each LE plus wiring delay between LEs, $t_{LE+wire}$, is $381 \text{ ps} + 246 \text{ ps} = 627 \text{ ps}$. The maximum number of LEs, N , is $Nt_{LE+wire} \leq 4.725 \text{ ns}$. Thus, $N = 7$.

Many ROMs and PLAs use *dynamic circuits* in place of pseudo-nMOS circuits. Dynamic gates turn the pMOS transistor ON for only part of the time, saving power when the pMOS is OFF and the result is not needed. Aside from this, dynamic and pseudo-nMOS memory arrays are similar in design and behavior.

5.6.3 Array Implementations*

To minimize their size and cost, ROMs and PLAs commonly use pseudo-nMOS (see Section 1.7.8) or dynamic circuits instead of conventional logic gates.

Figure 5.64(a) shows the dot notation for a 4×3 -bit ROM that performs the following functions: $X = A \oplus B$, $Y = \bar{A} + B$, and $Z = \bar{A}\bar{B}$. These are the same functions as those of Figure 5.51, with the address inputs renamed A and B and the data outputs renamed X , Y , and Z . The pseudo-nMOS implementation is given in Figure 5.64(b). Each decoder output is connected to the gates of the nMOS transistors in its row. Remember that in pseudo-nMOS circuits, the weak pMOS transistor pulls the output HIGH *only if* there is no path to GND through the pull-down (nMOS) network.

Pull-down transistors are placed at every junction without a dot. The dots from the dot notation diagram of Figure 5.64(a) are left visible in Figure 5.64(b) for easy comparison. The weak pull-up transistors pull the output HIGH for each wordline without a pull-down transistor. For example, when $AB = 11$, the 11 wordline is HIGH and transistors on X and Z

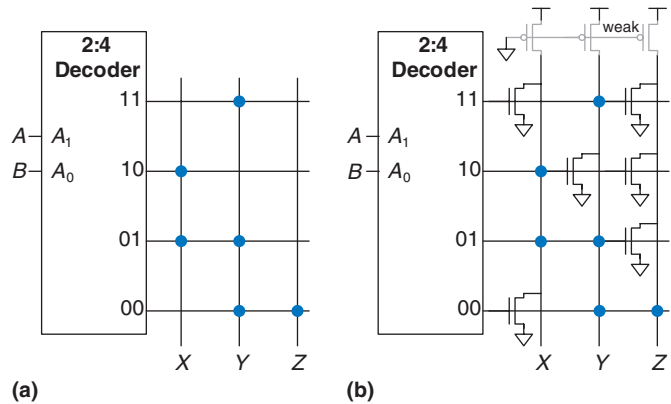


Figure 5.64 ROM implementation: (a) dot notation, (b) pseudo-nMOS circuit

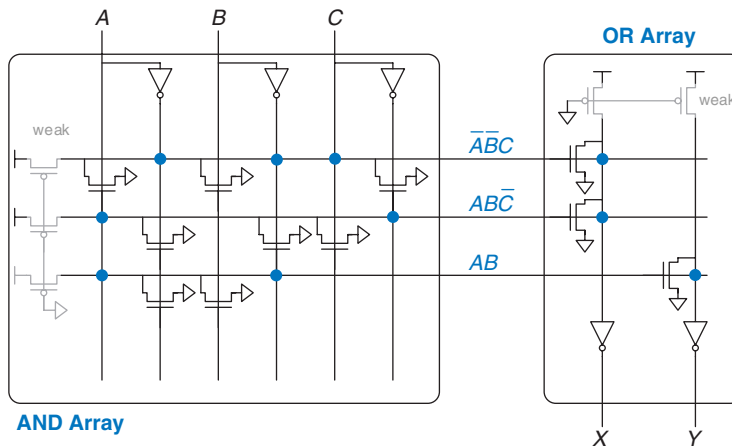


Figure 5.65 $3 \times 3 \times 2$ -bit PLA using pseudo-nMOS circuits

turn on and pull those outputs LOW. The Y output has no transistor connecting to the 11 wordline, so Y is pulled HIGH by the weak pull-up.

PLAs can also be built using pseudo-nMOS circuits, as shown in Figure 5.65 for the PLA from Figure 5.57. Pull-down (nMOS) transistors are placed on the *complement* of dotted literals in the AND array and on dotted rows in the OR array. The columns in the OR array are sent through an inverter before they are fed to the output bits. Again, the blue dots from the dot notation diagram of Figure 5.57 are left visible in Figure 5.65 for easy comparison.

5.7 SUMMARY

This chapter introduced digital building blocks used in many digital systems. These blocks include arithmetic circuits, such as adders, subtractors, comparators, shifters, multipliers, and dividers; sequential circuits, such as counters and shift registers; and arrays for memory and logic. The chapter also explored fixed-point and floating-point representations of fractional numbers. In Chapter 7, we use these building blocks to build a microprocessor.

Adders form the basis of most arithmetic circuits. A half adder adds two 1-bit inputs, A and B, and produces a sum and a carry out. A full adder extends the half adder to also accept a carry in. N full adders can be cascaded to form a carry propagate adder (CPA) that adds two N-bit numbers. This type of CPA is called a ripple-carry adder because the carry ripples through each of the full adders. Faster CPAs can be constructed using lookahead or prefix techniques.

A subtractor negates the second input and adds it to the first. A magnitude comparator subtracts one number from another and determines the relative value based on the sign or carry out of the result.

A multiplier forms partial products using AND gates, then sums these bits using full adders. A divider repeatedly subtracts the divisor from the partial remainder and checks the sign of the difference to determine the quotient bits. A counter uses an adder and a register to increment a running count.

Fractional numbers are represented using fixed-point or floating-point forms. Fixed-point numbers are analogous to decimals, and floating-point numbers are analogous to scientific notation. Fixed-point numbers use ordinary arithmetic circuits, whereas floating-point numbers require more elaborate hardware to extract and process the sign, exponent, and mantissa.

Large memories are organized into arrays of words. The memories have one or more ports to read and/or write the words. Volatile memories, such as SRAM and DRAM, lose their state when the power is turned off. SRAM is faster than DRAM but requires more transistors. A register file is a small multiported SRAM array. Nonvolatile memories, called ROMs, retain their state indefinitely. Despite their names, most modern ROMs can be written.

Arrays are also a regular way to build logic. Memory arrays can be used as LUTs to perform combinational functions. PLAs are composed of dedicated connections between configurable AND and OR arrays; they implement only combinational logic. FPGAs are composed of many small LUTs and registers; they implement combinational and sequential logic. The LUT contents and their interconnections can be configured to perform any logic function. Modern FPGAs are easy to reprogram and are large and cheap enough to build highly sophisticated digital systems, so they are widely used in low- and medium-volume commercial products, as well as in education.

Exercises

Exercise 5.1 What is the delay for the following types of 64-bit adders? Assume that each two-input gate delay is 150 ps and that a full adder delay is 450 ps.

- (a) a ripple-carry adder
- (b) a carry-lookahead adder with 4-bit blocks
- (c) a prefix adder

Exercise 5.2 Design two adders: a 64-bit ripple-carry adder and a 64-bit carry-lookahead adder with 4-bit blocks. Use only two-input gates. Each two-input gate is $15\text{ }\mu\text{m}^2$, has a 50 ps delay, and has 20 fF of total gate capacitance. You may assume that the static power is negligible.

- (a) Compare the area, delay, and power of the adders (operating at 100 MHz and 1.2 V).
- (b) Discuss the trade-offs between power, area, and delay.

Exercise 5.3 Explain why a designer might choose to use a ripple-carry adder instead of a carry-lookahead adder.

Exercise 5.4 Design the 16-bit prefix adder of [Figure 5.7](#) in an HDL. Simulate and test your module to prove that it functions correctly.

Exercise 5.5 The prefix network shown in [Figure 5.7](#) uses black cells to compute all of the prefixes. Some of the block propagate signals are not actually necessary. Design a “gray cell” that receives G and P signals for bits $i:k$ and $k-1:j$ but produces only G_{ij} , not P_{ij} . Redraw the prefix network, replacing black cells with gray cells wherever possible.

Exercise 5.6 The prefix network shown in [Figure 5.7](#) is not the only way to calculate all of the prefixes in logarithmic time. The *Kogge-Stone* network is another common prefix network that performs the same function using a different connection of black cells. Research Kogge-Stone adders and draw a schematic similar to [Figure 5.7](#) showing the connection of black cells in a Kogge-Stone adder.

Exercise 5.7 Recall that an N -input priority encoder has $\log_2 N$ outputs that encode which of the N inputs gets priority (see Exercise 2.36).

- (a) Design an N -input priority encoder that has delay that increases logarithmically with N . Sketch your design and give the delay of the circuit in terms of the delay of its circuit elements.
- (b) Code your design in an HDL. Simulate and test your module to prove that it functions correctly.

Exercise 5.8 Design the following comparators for 32-bit unsigned numbers. Sketch the schematics.

- (a) not equal
- (b) greater than or equal to
- (c) less than

Exercise 5.9 Consider the signed comparator of Figure 5.12.

- (a) Give an example of two 4-bit signed numbers A and B for which a 4-bit signed comparator correctly computes $A < B$.
- (b) Give an example of two 4-bit signed numbers A and B for which a 4-bit signed comparator incorrectly computes $A < B$.
- (c) In general, when does the N -bit signed comparator operate incorrectly?

Exercise 5.10 Modify the N -bit signed comparator of Figure 5.12 to correctly compute $A < B$ for all N -bit signed inputs A and B .

Exercise 5.11 Design the 32-bit ALU shown in Figure 5.15 using your favorite HDL. You can make the top-level module either behavioral or structural.

Exercise 5.12 Design the 32-bit ALU shown in Figure 5.17 using your favorite HDL. You can make the top-level module either behavioral or structural.

Exercise 5.13 Design the 32-bit ALU shown in Figure 5.18(a) using your favorite HDL. You can make the top-level module either behavioral or structural.

Exercise 5.14 Design the 32-bit ALU shown in Figure 5.18(b) using your favorite HDL. You can make the top-level module either behavioral or structural.

Exercise 5.15 Write a testbench to test the 32-bit ALU from Exercise 5.11. Then, use it to test the ALU. Include any test vector files necessary. Be sure to test enough corner cases to convince a reasonable skeptic that the ALU functions correctly.

Exercise 5.16 Repeat Exercise 5.15 for the ALU from Exercise 5.12.

Exercise 5.17 Repeat Exercise 5.15 for the ALU from Exercise 5.13.

Exercise 5.18 Repeat Exercise 5.15 for the ALU from Exercise 5.14.

Exercise 5.19 Build an *unsigned comparison unit* that compares two unsigned numbers A and B . The unit's input is the *Flags* signal (N , Z , C , V) from the ALU

of Figure 5.16, with the ALU performing subtraction: $A - B$. The unit's outputs are HS , LS , HI , and LO , which indicate that A is higher than or the same as (HS), lower than or the same as (LS), higher (HI), or lower (LO) than B .

- (a) Write minimal equations for HS , LS , HI , and LO in terms of N , Z , C , and V .
- (b) Sketch circuits for HS , LS , HI , and LO .

Exercise 5.20 Build a *signed comparison unit* that compares two signed numbers A and B . The unit's input is the *Flags* signal (N , Z , C , V) from the ALU of Figure 5.16, with the ALU performing subtraction: $A - B$. The unit's outputs are GE , LE , GT , and LT , which indicate that A is greater than or equal to (GE), less than or equal to (LE), greater than (GT), or less than (LT) B .

- (a) Write minimal equations for GE , LE , GT , and LT in terms of N , Z , C , and V .
- (b) Sketch circuits for GE , LE , GT , and LT .

Exercise 5.21 Design a shifter that always shifts a 32-bit input left by 2 bits. The input and output are both 32 bits. Explain the design in words and sketch a schematic. Implement your design in your favorite HDL.

Exercise 5.22 Design 4-bit left and right rotators. Sketch a schematic of each design. Implement your designs in your favorite HDL.

Exercise 5.23 Design an 8-bit left shifter using only 24 2:1 multiplexers. The shifter accepts an 8-bit input A and a 3-bit shift amount, $shamt_{2:0}$. It produces an 8-bit output Y . Sketch the schematic.

Exercise 5.24 Explain how to build any N -bit shifter or rotator using only $N \log_2 N$ 2:1 multiplexers.

Exercise 5.25 The *funnel shifter* in Figure 5.66 can perform any N -bit shift or rotate operation. It shifts a $2N$ -bit input right by k bits. The output Y is the N least significant bits of the result. The most significant N bits of the input are called B and the least significant N bits are called C . By choosing appropriate values of B , C , and k , the funnel shifter can perform any type of shift or rotate. Explain what these values should be in terms of A , $shamt$, and N for

- (a) logical right shift of A by $shamt$
- (b) arithmetic right shift of A by $shamt$
- (c) left shift of A by $shamt$
- (d) right rotate of A by $shamt$
- (e) left rotate of A by $shamt$

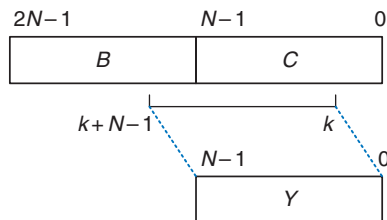


Figure 5.66 Funnel shifter

Exercise 5.26 Find the critical path for the unsigned 4×4 multiplier from Figure 5.21 in terms of an AND gate delay (t_{AND}) and a full adder delay (t_{FA}). What is the delay of an $N \times N$ multiplier built in the same way?

Exercise 5.27 Find the critical path for the unsigned 4×4 divider from Figure 5.22 in terms of a 2:1 mux delay (t_{MUX}), an adder delay (t_{FA}), and an inverter delay (t_{INV}). What is the delay of an $N \times N$ divider built in the same way?

Exercise 5.28 Design a multiplier that handles two's complement numbers.

Exercise 5.29 A *sign extension unit* extends a two's complement number from M to N ($N > M$) bits by copying the most significant bit of the input into the upper bits of the output (see Section 1.4.6). It receives an M -bit input A and produces an N -bit output Y . Sketch a circuit for a sign extension unit with a 4-bit input and an 8-bit output. Write the HDL for your design.

Exercise 5.30 A *zero extension unit* extends an unsigned number from M to N bits ($N > M$) by putting zeros in the upper bits of the output. Sketch a circuit for a zero extension unit with a 4-bit input and an 8-bit output. Write the HDL for your design.

Exercise 5.31 Compute $111001.000_2 / 001100.000_2$ in binary using the standard division algorithm from elementary school. Show your work.

Exercise 5.32 What is the range of numbers that can be represented by the following number systems?

- U12.12 format (24-bit unsigned fixed-point numbers with 12 integer bits and 12 fraction bits)
- 24-bit sign/magnitude fixed-point numbers with 12 integer bits and 12 fraction bits
- Q12.12 format (24-bit two's complement fixed-point numbers with 12 integer bits and 12 fraction bits)

Exercise 5.33 Express the following base 10 numbers in 16-bit fixed-point sign/magnitude format with eight integer bits and eight fraction bits. Express your answer in hexadecimal.

- (a) -13.5625
- (b) 42.3125
- (c) -17.15625

Exercise 5.34 Express the following base 10 numbers in 12-bit fixed-point sign/magnitude format with six integer bits and six fraction bits. Express your answer in hexadecimal.

- (a) -30.5
- (b) 16.25
- (c) -8.078125

Exercise 5.35 Express the base 10 numbers in Exercise 5.33 in Q8.8 format (16-bit fixed-point two's complement format with eight integer bits and eight fraction bits). Express your answer in hexadecimal.

Exercise 5.36 Express the base 10 numbers in Exercise 5.34 in Q6.6 format (12-bit fixed-point two's complement format with six integer bits and six fraction bits). Express your answer in hexadecimal.

Exercise 5.37 Express the base 10 numbers in Exercise 5.33 in IEEE 754 single-precision floating-point format. Express your answer in hexadecimal.

Exercise 5.38 Express the base 10 numbers in Exercise 5.34 in IEEE 754 single-precision floating-point format. Express your answer in hexadecimal.

Exercise 5.39 Convert the following Q4.4 (two's complement binary fixed-point numbers) to base 10. The implied binary point is explicitly shown to aid in your interpretation.

- (a) 0101.1000
- (b) 1111.1111
- (c) 1000.0000

Exercise 5.40 Repeat Exercise 5.39 for the following Q6.5 format (two's complement binary fixed-point) numbers.

- (a) 011101.10101
- (b) 100110.11010
- (c) 101000.00100

Exercise 5.41 When adding two floating-point numbers, the number with the smaller exponent is shifted. Why is this? Explain in words and give an example to justify your explanation.

Exercise 5.42 Add the following IEEE 754 single-precision floating-point numbers.

- (a) C0123456 + 81C564B7
- (b) D0B10301 + D1B43203
- (c) 5EF10324 + 5E039020

Exercise 5.43 Add the following IEEE 754 single-precision floating-point numbers.

- (a) C0D20004 + 72407020
- (b) C0D20004 + 40DC0004
- (c) (5FBE4000 + 3FF80000) + DFDE4000 (Why is the result counterintuitive? Explain.)

Exercise 5.44 Expand the steps in [Section 5.3.2](#) for performing floating-point addition to work for negative as well as positive floating-point numbers.

Exercise 5.45 Consider IEEE 754 single-precision floating-point numbers.

- (a) How many numbers can be represented by the IEEE 754 single-precision floating-point format? You need not count $\pm\infty$ or NaN.
- (b) How many additional numbers could be represented if $\pm\infty$ and NaN were not represented?
- (c) Explain why $\pm\infty$ and NaN are given special representations.

Exercise 5.46 Consider the following decimal numbers: 245 and 0.0625.

- (a) Write the two numbers using single-precision floating-point notation. Give your answers in hexadecimal.
- (b) Perform a magnitude comparison of the two 32-bit numbers from part (a). In other words, interpret the two 32-bit numbers as two's complement numbers and compare them. Does the integer comparison give the correct result?
- (c) You decide to come up with a new single-precision floating-point notation. Everything is the same as the IEEE 754 single-precision floating-point standard, except that you represent the exponent using two's complement

instead of a bias. Write the two numbers using your new standard. Give your answers in hexadecimal.

- (d) Does integer comparison work with your new floating-point notation from part (c)?
- (e) Why is it convenient for integer comparison to work with floating-point numbers?

Exercise 5.47 Design a single-precision floating-point adder using your favorite HDL. Before coding the design in an HDL, sketch a schematic of your design. Simulate and test your adder to prove to a skeptic that it functions correctly. You may consider positive numbers only and use round toward zero (truncate). You may also ignore the special cases given in [Table 5.4](#).

Exercise 5.48 In this problem, you will explore the design of a 32-bit floating-point multiplier. The multiplier has two 32-bit floating-point inputs and produces a 32-bit floating-point output. You may consider positive numbers only and use round toward zero (truncate). You may also ignore the special cases given in [Table 5.4](#).

- (a) Write the steps necessary to perform 32-bit floating-point multiplication.
- (b) Sketch the schematic of a 32-bit floating-point multiplier.
- (c) Design a 32-bit floating-point multiplier in an HDL. Simulate and test your multiplier to prove to a skeptic that it functions correctly.

Exercise 5.49 In this problem, you will explore the design of a 32-bit prefix adder.

- (a) Sketch a schematic of your design.
- (b) Design the 32-bit prefix adder in an HDL. Simulate and test your adder to prove that it functions correctly.
- (c) What is the delay of your 32-bit prefix adder from part (a)? Assume that each two-input gate delay is 100 ps.
- (d) Design a pipelined version of the 32-bit prefix adder. Sketch the schematic of your design. How fast can your pipelined prefix adder run? You may assume a sequencing overhead ($t_{pcq} + t_{setup}$) of 80 ps. Make the design run as fast as possible.
- (e) Design the pipelined 32-bit prefix adder in an HDL.

Exercise 5.50 An incrementer adds 1 to an N -bit number. Build an 8-bit incrementer using half adders.

Exercise 5.51 Build a 32-bit synchronous *Up/Down counter*. The inputs are *Reset* and *Up*. When *Reset* is 1, the outputs are all 0. Otherwise, when *Up* = 1, the circuit counts up, and when *Up* = 0, the circuit counts down.

Exercise 5.52 Design a 32-bit counter that adds 4 at each clock edge. The counter has reset and clock inputs. Upon reset, the counter output is all 0.

Exercise 5.53 Modify the counter from Exercise 5.52 such that the counter will either increment by 4 or load a new 32-bit value, *D*, on each clock edge, depending on a control signal *Load*. When *Load* = 1, the counter loads the new value *D*. Otherwise, it increments by 4.

Exercise 5.54 An *N*-bit *Johnson counter* consists of an *N*-bit shift register with a reset signal. The output of the shift register (S_{out}) is inverted and fed back to the input (S_{in}). When the counter is reset, all of the bits are cleared to 0.

- Show the sequence of outputs, $Q_{3:0}$, produced by a 4-bit Johnson counter starting immediately after the counter is reset.
- How many cycles elapse until an *N*-bit Johnson counter repeats its sequence? Explain.
- Design a decimal counter using a 5-bit Johnson counter, ten AND gates, and inverters. The decimal counter has a clock, a reset, and ten one-hot outputs $Y_{9:0}$. When the counter is reset, Y_0 is asserted. On each subsequent cycle, the next output should be asserted. After ten cycles, the counter should repeat. Sketch a schematic of the decimal counter.
- What advantages might a Johnson counter have over a conventional counter?

Exercise 5.55 Write the HDL for a 4-bit scannable flip-flop like the one shown in Figure 5.39. Simulate and test your HDL module to prove that it functions correctly.

Exercise 5.56 The English language has a good deal of redundancy that allows us to reconstruct garbled transmissions. Binary data can also be transmitted in redundant form to allow error correction. For example, the number 0 could be coded as 00000 and the number 1 could be coded as 11111. The value could then be sent over a noisy channel that might flip up to two of the bits. The receiver could reconstruct the original data because a 0 will have at least three of the five received bits as 0's; similarly, a 1 will have at least three 1's.

- Propose an encoding to send 00, 01, 10, or 11 encoded using five bits of information such that all errors that corrupt one bit of the encoded data

can be corrected. Hint: the encodings 00000 and 11111 for 00 and 11, respectively, will not work.

- (b) Design a circuit that receives your five-bit encoded data and decodes it to 00, 01, 10, or 11, even if one bit of the transmitted data has been changed.
- (c) Suppose you wanted to change to an alternative 5-bit encoding. How might you implement your design to make it easy to change the encoding without having to use different hardware?

Exercise 5.57 Flash EEPROM, simply called Flash memory, is a fairly recent invention that has revolutionized consumer electronics. Research and explain how Flash memory works. Use a diagram illustrating the floating gate. Describe how a bit in the memory is programmed. Properly cite your sources.

Exercise 5.58 The extraterrestrial life project team has just discovered aliens living on the bottom of Mono Lake. They need to construct a circuit to classify the aliens by potential planet of origin based on measured features available from the NASA probe: greenness, brownness, sliminess, and ugliness. Careful consultation with xenobiologists leads to the following conclusions:

- If the alien is green and slimy or ugly, brown, and slimy, it might be from Mars.
- If the critter is ugly, brown, and slimy, or green and neither ugly nor slimy, it might be from Venus.
- If the beastie is brown and neither ugly nor slimy or is green and slimy, it might be from Jupiter.

Note that this is an inexact science; for example, a life form which is mottled green and brown and is slimy but not ugly might be from either Mars or Jupiter.

- (a) Program a $4 \times 4 \times 3$ PLA to identify the alien. You may use dot notation.
- (b) Program a 16×3 ROM to identify the alien. You may use dot notation.
- (c) Implement your design in an HDL.

Exercise 5.59 Implement the following functions using a single 16×3 ROM. Use dot notation to indicate the ROM contents.

- (a) $X = AB + B\overline{C}D + \overline{A}B$
- (b) $Y = AB + BD$
- (c) $Z = A + B + C + D$

Exercise 5.60 Implement the functions from Exercise 5.59 using a $4 \times 8 \times 3$ PLA. You may use dot notation.

Exercise 5.61 Specify the size of a ROM that you could use to program each of the following combinational circuits. Is using a ROM to implement these functions a good design choice? Explain why or why not.

- (a) a 16-bit adder/subtractor with C_{in} and C_{out}
- (b) an 8×8 multiplier
- (c) a 16-bit priority encoder (see Exercise 2.36)

Exercise 5.62 Consider the ROM circuits in Figure 5.67. For each row, can the circuit in column I be replaced by an equivalent circuit in column II by proper programming of the latter's ROM?

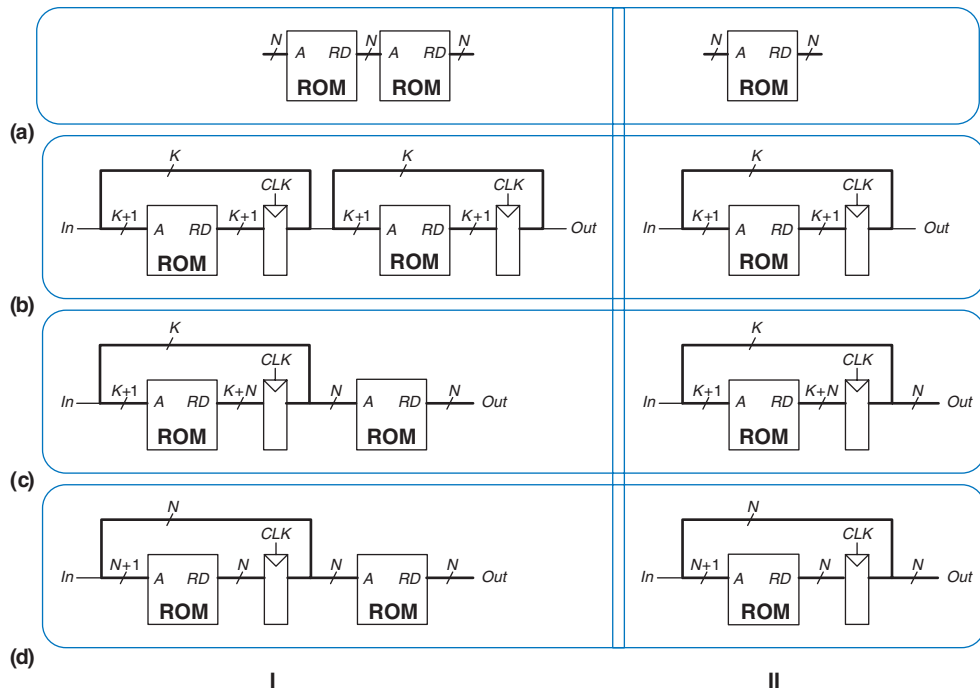


Figure 5.67 ROM circuits

Exercise 5.63 How many Cyclone IV FPGA LEs are required to perform each of the following functions? Show how to configure one or more LEs to perform the

function. You should be able to do this by inspection, without performing logic synthesis.

- (a) the combinational function from Exercise 2.13(c)
- (b) the combinational function from Exercise 2.17(c)
- (c) the two-output function from Exercise 2.24
- (d) the function from Exercise 2.35
- (e) a four-input priority encoder (see Exercise 2.36)

Exercise 5.64 Repeat Exercise 5.63 for the following functions:

- (a) an eight-input priority encoder (see Exercise 2.36)
- (b) a 3:8 decoder
- (c) a 4-bit carry propagate adder (with no carry in or out)
- (d) the FSM from Exercise 3.22
- (e) the Gray code counter from Exercise 3.27

Exercise 5.65 Consider the Cyclone IV LE shown in Figure 5.60. According to the datasheet, it has the timing specifications given in Table 5.7.

- (a) What is the minimum number of Cyclone IV LEs required to implement the FSM of Figure 3.26?
- (b) Without clock skew, what is the fastest clock frequency at which this FSM will run reliably?
- (c) With 3 ns of clock skew, what is the fastest frequency at which the FSM will run reliably?

Table 5.7 Cyclone IV timing

Name	Value (ps)
t_{pcq}, t_{ccq}	199
t_{setup}	76
t_{hold}	0
t_{pd} (per LE)	381
t_{wire} (between LEs)	246
t_{skew}	0

Exercise 5.66 Repeat Exercise 5.65 for the FSM of [Figure 3.31\(b\)](#).

Exercise 5.67 You would like to use an FPGA to implement an M&M sorter with a color sensor and motors to put red candy in one jar and green candy in another. The design is to be implemented as an FSM using a Cyclone IV FPGA. According to the data sheet, the FPGA has timing characteristics shown in [Table 5.7](#). You would like your FSM to run at 100 MHz. What is the maximum number of LEs on the critical path? What is the fastest speed at which the FSM will run?

Interview Questions

The following exercises present questions that have been asked at interviews for digital design jobs.

Question 5.1 What is the largest possible result of multiplying two unsigned N -bit numbers?

Question 5.2 *Binary coded decimal (BCD)* representation uses four bits to encode each decimal digit. For example, 42_{10} is represented as 01000010_{BCD} . Explain in words why processors might use BCD representation.

Question 5.3 Design hardware to add two 8-bit unsigned BCD numbers (see Question 5.2). Sketch a schematic for your design, and write an HDL module for the BCD adder. The inputs are A , B , and C_{in} , and the outputs are S and C_{out} . C_{in} and C_{out} are 1-bit carries and A , B , and S are 8-bit BCD numbers.

